Guía esencial de base de datos para la defensa (2 h 30)

Oct 14, 2026 · @juan carlos bodoque III

1. Cómo usar esta guía en 2 h 30

Esta es la versión comprimida de la guía completa de base de datos: conserva solo lo que más probablemente te preguntarán y lo que necesitas para explicar el script sin leerlo. Tiene unas 18.000 palabras en lugar de 29.000 y mantiene el banco de preguntas, con respuestas cortas. La lectura son unos 75 minutos; el resto del tiempo es para **practicar en voz alta**, que es lo que realmente prepara para la defensa.

Bloque	Secciones	Lectura	Práctica	Al terminar deberías poder…
1. El mapa	2	5 min	4 min	Decir qué hay en la base y cómo se reparte la lógica
2. Las tablas	3	10 min	7 min	Dibujar de memoria `Empleados` y `UnidadesActivo` con sus relaciones
3. Vistas y funciones	4	6 min	5 min	Explicar la `Alerta` de los préstamos y el descuento por ausencias
4. Procedimientos	5	9 min	8 min	Explicar `sp_RegistrarMovimientoConsumible` línea por línea
5. Triggers y auditoría	6	7 min	7 min	Contar cómo se registra un cambio y quién lo hizo
6. Base y C#	7	4 min	4 min	Seguir una llamada desde el botón hasta la tabla
7. Flujos	8	5 min	7 min	Narrar préstamo, consumible y planilla con las cuatro preguntas
8. Leer un script	9	6 min	6 min	Leer un `CREATE TABLE` y la cabecera del script sin dudar
9. Preguntas	10	20 min	10 min	Responder cada pregunta con tus palabras, sin mirar
10. Repaso final	11	3 min	17 min	Hacer la demostración en SSMS y decir los límites del diseño

Suma 150 minutos justos: aprovecha los descansos entre bloques y, si vas sobrado, usa el tiempo extra en las preguntas marcadas con ★.

Qué se recortó. La descripción tabla por tabla de los módulos secundarios, la explicación una por una de las 27 vistas y de los procedimientos menos preguntados (queda la lista por familias), el detalle del SQL dinámico del generador de triggers y las preguntas menos probables (las 73 de la guía completa quedaron condensadas en 40, fusionando las que se parecen). Si te preguntan algo de eso, está en la guía completa; con lo que queda aquí puedes defender la base.

Cómo estudiar. Lee una sección, ciérrala y cuenta en voz alta lo que entendiste como si hablaras con el profesor. Si te trabas en un punto, reléelo solo a él. No memorices números de error ni nombres de las 27 vistas; entiende **por qué** se hace cada cosa, que es lo que se

pregunta. Para cualquier objeto de la base (tabla, vista, procedimiento o trigger) usa las **cuatro preguntas**: ¿qué problema resuelve?, ¿cómo lo hace?, ¿dónde se ve en el programa? y ¿qué pasa si falla o alguien intenta saltarlo?

2. El mapa de la base

Tu base de datos no solo guarda información: también decide qué está permitido y qué no. Entender eso es la clave de toda la defensa.

2.1 Qué contiene el script

El archivo `BaseDeDatos_Completa.sql` tiene cinco tipos de objetos y cada uno resuelve un problema distinto.

Pieza	Cantidad	Qué problema resuelve	Ejemplo real
Tablas	30	Guardar los datos de forma ordenada y protegida	`Empleados`, `UnidadesActivo`, `Prestamos`
Funciones	6	Cálculos reutilizables que devuelven un valor o una tabla	`fn_DiasHabiles`, `fn_PeriodoCerrado`
Vistas	27	Consultas guardadas con nombre, listas para mostrar en pantalla	`vw_Prestamos`, `vw_StockConsumibles`
Procedimientos almacenados	34	Operaciones completas con reglas de negocio	`sp_RegistrarPrestamo`, `sp_GenerarPlanilla`
Triggers	20	Código que se dispara solo cuando cambia una tabla	`trg_Aud_Empleados`, `trg_Auditoria_Inmutable`

De los 20 triggers, 17 son de registro de actividad (uno por tabla auditada), uno protege el registro para que nadie lo borre y dos llevan el historial del inventario. Una frase para memorizar: **las tablas guardan, las vistas muestran, los procedimientos deciden, las funciones calculan y los triggers vigilan.**

2.2 Los siete grupos de tablas

Las 30 tablas se entienden por grupos, y cada grupo corresponde a una parte del menú del programa.

Grupo	Tablas	Para qué sirve
Personal y acceso	`Roles`, `Usuarios`, `Cargos`, `EstadosEmpleado`, `Empleados`, `EmpleadoFotos`, `EmpleadoMaterias`	Quién trabaja en la institución y quién puede entrar al sistema
Académico	`Grados`, `Secciones`, `Materias`, `AsignacionesDocente`	Qué docente da qué materia en qué grupo y cuántas horas
Planillas	`Planillas`, `PagosPlanilla`, `PeriodosPlanilla`, `PeriodosPlanillaHistorial`, `Ausencias`	Salarios, pagos mensuales, cierre de mes y descuentos por faltas
Activos	`Categorias`, `Estados`, `GestionActivos`, `UnidadesActivo`, `ReportesDanio`, `Prestamos`, `MantenimientosActivo`, `AuditoriaUnidades`, `AuditoriaActivos`	El inventario físico, su estado, sus préstamos y su mantenimiento
Salones	`Salones`	Dónde está cada cosa; incluye la bodega
Consumibles	`Consumibles`, `MovimientosConsumible`	Papel, marcadores y similares, con su kardex
Sistema	`ConfiguracionEmpresa`, `Auditoria`	Datos de la institución y registro de quién hizo qué

2.3 La idea que ordena todo: las reglas viven en la base

Hay dos lugares donde puede vivir una regla: en la pantalla (C#) o en la base de datos. Tu proyecto pone las reglas importantes en la base, porque la pantalla puede cambiar, pero los datos son lo que no se puede dañar. De ahí salen **tres barreras** que se repiten en todo el sistema:

1. **La pantalla** ayuda: valida lo evidente y avisa junto al campo. Es comodidad.
2. **El procedimiento almacenado** garantiza: repite las validaciones importantes en el servidor y lanza un mensaje claro con `THROW`.

3. **La restricción de la tabla** (`CHECK`, `UNIQUE`, llave foránea) es la red final: protege incluso contra un `UPDATE` escrito a mano en Management Studio.

Un ejemplo concreto: el campo numérico de «Asignar equipo» no deja escribir más de lo disponible (pantalla), pero aunque eso fallara, `sp_AsignarUnidades` vuelve a contar las unidades disponibles y rechaza el pedido con el error 50514 (procedimiento). **La pantalla ayuda al usuario; la base garantiza la verdad.**

Hay un matiz honesto que conviene que sepas: las restricciones de las tablas protegen siempre, pero las reglas más elaboradas, como «una baja necesita motivo», viven en los procedimientos, y un `UPDATE` escrito a mano sobre la tabla se las salta. Por eso el sistema siempre pasa por los procedimientos para las operaciones con reglas serias.

3. Las tablas: llaves, restricciones y cómo leerlas

Las tablas son el cimiento: si están bien diseñadas, los datos incorrectos ni siquiera pueden entrar. Aquí tienes los conceptos que lo explican casi todo y después las tablas que más te van a preguntar.

3.1 Tabla, fila, columna y llaves

Una tabla guarda un tipo de cosa (`Empleados` guarda personas), cada fila es un caso concreto y cada columna una característica. El **tipo** de la columna ya es una primera protección: el dinero va en `DECIMAL(10,2)` porque los decimales exactos no se redondean mal como los `FLOAT`, y una fecha sin hora va en `DATE`.

La **llave primaria** (`PRIMARY KEY`) identifica cada fila sin ambigüedad: no se repite ni queda vacía. Casi todas tus tablas usan `IdAlgo INT IDENTITY(1,1)`: el motor numera solo, 1, 2, 3, sin repetir (y sin reutilizar números: si borras el 7, el siguiente sigue siendo el 8). Algunas llaves usan dos columnas: `EmpleadoMaterias` tiene `PRIMARY KEY (IdEmpleado, IdMateria)`, así que un docente no puede tener «Matemática» dos veces.

La **llave foránea** (`FOREIGN KEY`) es una columna que apunta a la llave primaria de otra tabla: `Empleados.IdCargo` apunta a `Cargos.IdCargo`. Eso garantiza la **integridad referencial** en las dos direcciones: no puedes asignar un cargo que no existe ni borrar un cargo que aún usa alguien (quedaría una fila huérfana). Lo impide el motor, no el programa, así que se cumple aunque alguien escriba SQL directo.

- **Uno a muchos:** un cargo tiene muchos empleados. Es la más común y se resuelve con una llave foránea en el lado «muchos».
- **Muchos a muchos:** un docente da varias materias y una materia la dan varios docentes. Se resuelve con una tabla intermedia (`EmpleadoMaterias`) cuya llave primaria es la pareja de dos llaves foráneas.

`ON DELETE CASCADE` hace que al borrar el padre se borren solas las filas hijas. Solo está en cinco relaciones, y todas son datos sin sentido propio: la foto, las materias y la carga de un empleado, y las unidades y reportes de un equipo del catálogo. **No** está en `PagosPlanilla` ni en `Prestamos`, porque son historial: por eso un empleado con pagos no se puede borrar y el programa lo da de baja.

3.2 Restricciones: las reglas que guarda la tabla

Son las reglas más baratas y más fuertes, porque el motor las aplica siempre.

Restricción	Qué dice	Ejemplo de tu script
`NOT NULL`	La columna no puede quedar vacía	`Usuarios.Clave`
`DEFAULT`	Valor que se pone si nadie indica otro	`Prestamos.Estado` empieza en 'Activo'
`UNIQUE`	Ningún valor se repite	`Empleados.Nombre`, `Usuarios.Usuario`
`CHECK`	Una condición que cada fila debe cumplir	`Salones.Capacidad` entre 1 y 500
`FOREIGN KEY`	El valor debe existir en otra tabla	`Prestamos.IdUnidad`

El `CHECK` más interesante es el de `PagosPlanilla`: obliga a que el salario base sea mayor que cero y a que el descuento no supere base más bono, así que un pago imposible no puede existir. **Un detalle que sorprende:** un `CHECK` solo rechaza cuando la condición da *falso*; con `NULL` da «desconocido» y la fila pasa. Para exigir que un dato exista hay que declarar la columna `NOT NULL`. Un `CHECK` solo ve la fila que se guarda: no puede consultar otras filas ni tablas, y por eso el tope de 40 horas semanales de un docente (que suma varias filas) lo valida un procedimiento.

Todas las restricciones llevan un **nombre con prefijo**: `PK_` primaria, `FK_` foránea, `UQ_` única, `CK_` chequeo, `DF_` valor por defecto y `UX_` índice único creado aparte. El nombre aparece en el mensaje de error y el programa C# lo usa para avisar al usuario con precisión.

3.3 Índices únicos filtrados y columnas calculadas

A veces la unicidad solo aplica a una parte de las filas. «Un empleado tiene un solo pago por mes» es falso si cuentas los pagos anulados, porque se puede anular y volver a registrar. La solución es un índice único con `WHERE`:

```
CREATE UNIQUE INDEX UX_PagosPlanilla_Empleado_Periodo
    ON dbo.PagosPlanilla (IdEmpleado, Periodo) WHERE ([Anulado] = 0);
```

Los pagos anulados quedan fuera del índice y no estorban. El mismo truco aparece tres veces más: `UX_Prestamos_UnidadActiva` (una unidad no puede tener dos préstamos activos, aunque en su vida tenga muchos ya devueltos), `UX_ReportesDanio_UnidadAbierta` (un solo reporte abierto por unidad) y `UX_Empleados_DUI` (`WHERE DUI IS NOT NULL`: el DUI es opcional, y un `UNIQUE` normal permitiría un solo empleado sin DUI porque trata los `NULL` como iguales). Si se viola uno de estos índices, SQL Server devuelve el error **2601**; si se viola un `UNIQUE`, el **2627**.

Una **columna calculada** no se escribe nunca: `SalarioNeto AS (SalarioBase + Bono - Descuento) PERSISTED` la calcula y guarda el motor, así que el neto no puede quedar desincronizado con sus componentes.

3.4 Normalización, y dónde se rompe a propósito

Normalizar es guardar cada dato en un solo lugar. Por eso existe `Cargos` (el texto «Coordinador Académico» se guarda una vez) y por eso el inventario tiene dos niveles: `GestionActivos` es el **catálogo** («Pupitre») y `UnidadesActivo` son las **piezas físicas**, cada una con su código y su estado. Si el estado viviera en el catálogo, todos los pupitres tendrían el mismo estado, que fue el error del diseño original.

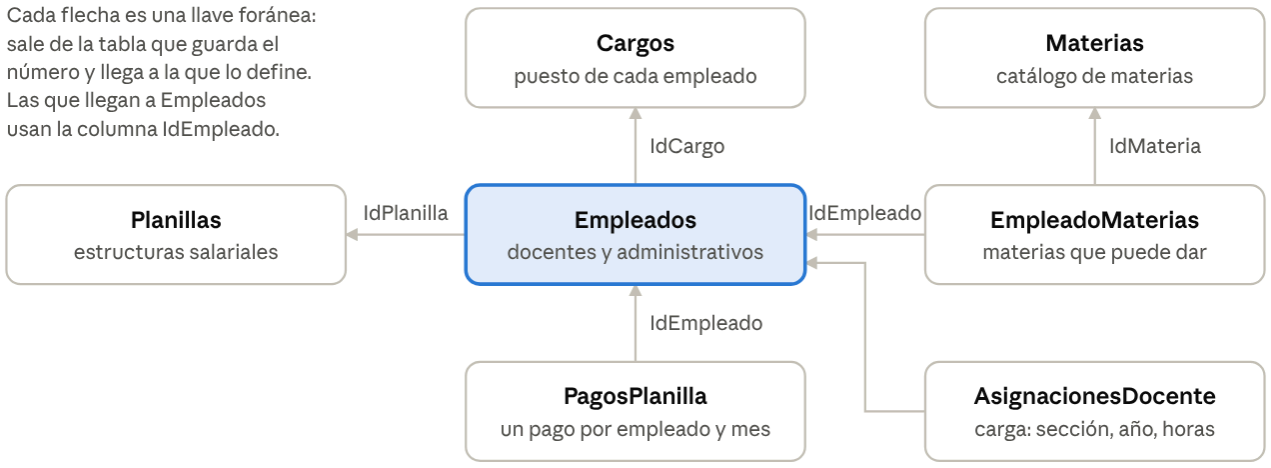
Hay desviaciones deliberadas y conviene saberlas decir: `PagosPlanilla` **copia** el salario base, el bono y el descuento de la estructura salarial, como una fotografía de ese mes (si en abril sube el salario de la estructura, el pago de marzo no debe cambiar, igual que una factura guarda el precio del día). `Prestamos` también guarda el nombre del responsable y el código de inventario. Regla: se normaliza para evitar contradicciones al guardar y se repite cuando el dato es una foto histórica.

3.5 Empleados y Usuarios: personas y cuentas

Son dos cosas distintas: `Empleados` es el personal; `Usuarios` son las cuentas que pueden entrar al sistema. Un conserje es empleado aunque nunca tenga usuario. En `Empleados` hay tres protecciones que debes saber explicar: `UNIQUE (Nombre)` (no hay dos empleados con el mismo nombre), el `CHECK` del DUI más su índice único filtrado, y llaves foráneas a `Cargos`, `EstadosEmpleado` y `Planillas`. El estado 2 es Inactivo y es lo que usa la baja lógica. Una precisión honesta: en la tabla solo `Nombre` es obligatorio; que el empleado tenga cargo, estado y planilla lo exige el formulario porque esas columnas aceptan `NULL`.

En `Usuarios`, `Clave` guarda un hash BCrypt, nunca la contraseña. También están `Activo` (desactivar sin borrar) y `DebeCambiarClave` (forzar una clave nueva tras un restablecimiento).

Empleados es el centro del personal, los pagos y la carga



relaciones de Empleados · 7 tablas, 6 llaves foráneas

Las tablas que apuntan a `Empleados` (pagos, materias y carga) no pueden tener una fila sin un empleado real. `Cargos` y `Planillas` son catálogos a los que el empleado hace referencia. `AsignacionesDocente` también apunta a `Materias` y a `Secciones`, y `EmpleadoFotos` también cuelga de `Empleados`; se dejaron fuera para que el dibujo se lea de un vistazo.

3.6 Salones e inventario

`Salones` tiene una peculiaridad: `IdSalon` **no** es `IDENTITY`; `sp_GuardarSalon` calcula el siguiente como máximo más uno, protegido con `UPDLOCK, HOLDLOCK` para que dos administradores no obtengan el mismo número. Tiene `CHECK` para el tipo (siete valores permitidos) y la capacidad (1 a 500). La **bodega** no es una tabla aparte: es un salón con `Tipo = 'Bodega'` que `fn_IdBodega()` encuentra. Todo el equipo nuevo entra ahí y después se asigna a los salones.

El inventario es la parte más valiosa del diseño: `GestionActivos` (catálogo) y `UnidadesActivo` (piezas físicas, con código tipo `ACT-0004-001`, estado, salón, costo, proveedor y, si aplica, fecha y motivo de baja), en relación de uno a muchos con `ON DELETE CASCADE`. El código de inventario tiene un índice único filtrado (no se repite, pero puede estar vacío) y `CostoUnitario` un `CHECK` para que nunca sea negativo.

Cada pieza física cuelga de un tipo de equipo y vive en un salón



relaciones de UnidadesActivo · 7 tablas, 7 llaves foráneas

Una unidad pertenece a un tipo de equipo (`GestionActivos`) y está en un salón, y por eso el estado, la ubicación y el historial son de cada pieza y no del tipo. Los préstamos, los reportes de daño y los mantenimientos cuelgan de la unidad. `Salones` y `Prestamos` también apuntan a `Empleados` , que es donde el inventario y el personal se encuentran.

3.7 Préstamos, daños y mantenimientos

Tabla	Regla que la protege	Cómo se aplica
`Prestamos`	Una unidad no puede tener dos préstamos activos	Índice único filtrado `WHERE Estado = 'Activo'`
`Prestamos`	El tipo de responsable es Docente, Personal administrativo o Alumno	`CHECK` en `TipoResponsable`
`ReportesDanio`	Una unidad no puede tener dos reportes abiertos	Índice único filtrado `WHERE Estado = 'Abierto'`
`ReportesDanio`	La descripción no puede ser solo espacios	`CHECK` con `LTRIM` y `RTRIM`
`MantenimientosActivo`	El tipo es Preventivo o Correctivo y el costo no es negativo	Dos `CHECK`

En `Prestamos` , si el responsable es Docente o Personal administrativo, `IdEmpleado` apunta a un empleado real; si es Alumno, se guarda solo el nombre como texto. Esa regla la aplica `sp_RegistrarPrestamo` .

3.8 Consumibles, planillas y auditoría

Consumibles. `Consumibles` guarda `StockActual` y `StockMinimo`, ambos con `CHECK` para que no sean negativos. `MovimientosConsumible` es el **kardex**: cada entrada o salida es una fila con cantidad, motivo, destino y el saldo resultante, para que el historial se lea sin recalcular.

Planillas. `Planillas` es el **catálogo de estructuras salariales** (base, bono, descuento) y cada empleado tiene asignada una; `PagosPlanilla` es lo que realmente se paga cada mes. En los pagos hay cuatro protecciones: el período siempre es el día 1 del mes, los montos son coherentes, `SalarioNeto` es calculada y existe un único pago vigente por empleado y mes. `PeriodosPlanilla` guarda qué meses están cerrados y `PeriodosPlanillaHistorial` anota quién cerró o reabrió cada uno. `Ausencias` alimenta el descuento: tiene tipo (Falta, Permiso, Incapacidad), si está justificada, si es con goce de sueldo, y se anula con una bandera `Anulada` en lugar de borrarse.

Auditoria registra quién, cuándo, en qué tabla y qué cambió, con el contenido antes y después como texto JSON. Un `CHECK` limita la operación a cuatro letras: `I` insertar, `U` modificar, `D` borrar y `A` acceso. Tiene tres índices para buscar rápido por fecha, usuario o tabla.

Datos de ejemplo. El script carga datos iniciales (roles, estados, cargos, grados) y de ejemplo (50 empleados, 8 materias, 5 estructuras salariales, 13 salones más la bodega, 31 tipos de equipo y 692 unidades), que se insertan solo si faltan. El script **no** crea usuarios: el primer administrador sale del asistente de primer uso del programa.

4. Vistas y funciones: consultar sin repetir trabajo

Las funciones calculan y las vistas presentan. Ninguna de las dos guarda datos propios: siempre trabajan sobre las tablas, así que lo que muestran está al día.

4.1 Vistas

Una vista es una **consulta guardada con nombre**. Escribir `SELECT * FROM vw_Prestamos` es como ejecutar la consulta completa que está dentro, con sus uniones y cálculos; la vista no guarda copia de nada. No es más rápida que escribir el `SELECT` a mano: su valor es la organización. Sirve para tres cosas:

- **Simplificar.** La pantalla de préstamos necesita el nombre del equipo, el aula, el estado y una alerta; son varias uniones que la vista hace una vez. Además traduce números a palabras (la tabla guarda `IdEstado = 3`, la vista entrega `Estado = 'Malo'`).
- **Centralizar lógica.** Una regla calculada en la vista se ve igual en todas las pantallas.

- **Ordenar el acceso.** La pantalla **lee vistas y escribe mediante procedimientos**, así que cada cosa tiene su camino.

El ejemplo más claro es la columna `Alerta` de `vw_Prestamos`:

```
CASE WHEN p.Estado = 'Devuelto'                       THEN 'Devuelto'
     WHEN p.FechaLimite < GETDATE()                   THEN 'Atrasado'
     WHEN p.FechaLimite < DATEADD(DAY, 2, GETDATE()) THEN 'Por vencer'
     ELSE 'En curso' END AS Alerta
```

El `CASE` se evalúa de arriba abajo y se queda con la primera condición que se cumple, y **el orden importa**: un préstamo devuelto hace un mes tiene la fecha límite vencida, así que si el atraso se preguntara primero aparecería atrasado para siempre. Como usa `GETDATE()`, un préstamo pasa de «En curso» a «Por vencer» y luego a «Atrasado» sin que nadie actualice nada: **la alerta no se guarda, se calcula al mirar**. `vw_StockConsumibles` sigue la misma idea (`Agotado` si el stock es cero, `Stock bajo` si es menor o igual al mínimo, `Normal` en otro caso) y la pantalla solo traduce esa palabra a un color.

`JOIN` y `LEFT JOIN` en las vistas. La pregunta que decide es «¿puede existir esta fila sin la otra?». En `vw_UnidadesDetalle` el equipo, el salón y el estado son obligatorios (`JOIN`), pero la categoría puede ser nula (`LEFT JOIN`). En `vw_EmpleadosDetalle` un empleado puede no tener cargo, estructura salarial ni pago del mes, y todos son `LEFT JOIN` para que no desaparezca de la lista; para el pago del mes actual usa `OUTER APPLY` con `TOP 1` («el último pago vigente de este empleado en el mes») y `ISNULL` lo muestra como «Sin registrar». Al contar con un `LEFT JOIN`, `COUNT(u.IdUnidad)` ignora los `NULL` (un salón sin equipos da 0) y `COUNT(*)` daría 1.

Las 27 vistas por familias. No hace falta memorizarlas; basta saber a cuál pertenece cada una.

Familia	Vistas	Qué hacen
Detalle	`vw_UnidadesDetalle`, `vw_ActivosDetalle`, `vw_EmpleadosDetalle`, `vw_PagosPlanillaDetalle`, `vw_Prestamos`, `vw_Mantenimientos`, `vw_ReportesDanio`, `vw_AusenciasDetalle`	Unen tablas y traen nombres en lugar de números

Familia	Vistas	Qué hacen
Resúmenes agrupados	`vw_ResumenActivosPorEstado`, `vw_ResumenActivosPorCategoria`, `vw_ResumenEmpleadosPorEstado`, `vw_ResumenPlanillaPorCargo` y otras	Agrupan y cuentan; hoy ninguna pantalla las llama (el inicio hace sus propias consultas), sirven para reportes directos
Disponibilidad y stock	`vw_StockBodega`, `vw_StockConsumibles`, `vw_KardexConsumibles`, `vw_CargaDocente`	Responden «cuánto hay» o «cuánto queda»
Auditoría	`vw_Auditoria`, `vw_AuditoriaUnidadesDetalle`, `vw_AuditoriaActivosDetalle`, `vw_HistorialMovimientos`	Convierten el historial técnico en frases legibles

Dos que conviene saber explicar. **`vw_StockBodega`** cuenta las unidades que cumplen tres condiciones a la vez: estar en la bodega (`u.IdSalon = dbo.fn_IdBodega()`), no tener un estado problemático (`NOT IN ('Malo', 'En reparación', 'Mantenimiento Pendiente', 'Dado de baja')`) y no estar prestada ahora (`NOT EXISTS` de un préstamo activo); así «Disponibles» es exactamente lo asignable. **`vw_Auditoria`** alimenta la pantalla de Registro de actividad: traduce la tabla técnica a un módulo en español, decide la acción en lenguaje humano («Creó», «Modificó», «Anuló», «Inició sesión») leyendo el JSON con `JSON_VALUE`, y arma el resumen con `fn_AuditoriaCambios` y `STRING_AGG`. Es un buen ejemplo de una base que entrega el dato ya listo para mostrar.

4.2 Funciones

Una función recibe valores y devuelve un resultado, y está pensada para usarse **dentro** de otra consulta. Tu script tiene seis: cinco **escalares** (devuelven un solo valor), que son `fn_DiasHabiles`, `fn_IdBodega`, `fn_PeriodoCerrado`, `fn_DiasDescontados` y `fn_DescuentoAusencias`, y una **de tabla**, `fn_AuditoriaCambios`, que recibe el número de un registro de auditoría y devuelve una fila por cada campo que cambió. Una función no puede modificar tablas, y eso es una garantía a favor: se puede usar en cualquier consulta sabiendo que solo calcula.

	Vista	Función	Procedimiento
¿Recibe parámetros?	No (consulta fija)	Sí	Sí

	Vista	Función	Procedimiento
¿Cómo se usa?	`SELECT ... FROM vw_X`	Dentro de otra consulta (`WHERE x = dbo.fn_Y()`)	Con `EXEC`
¿Puede modificar datos y abrir transacciones?	No	No	Sí

La mejor forma de entenderlas es ver cómo se encadenan para calcular el **descuento por ausencias**:

1. `fn_DiasHabiles(inicio, fin)` cuenta los días de lunes a viernes entre dos fechas.
2. `fn_DiasDescontados(empleado, período)` suma los días hábiles que caen dentro del mes en las faltas injustificadas y los permisos sin goce de sueldo (una falta justificada o un permiso con goce no descuentan; una ausencia que cruza de mes solo cuenta la parte del mes del pago).
3. `fn_DescuentoAusencias(empleado, período, salario)` divide el salario entre los días hábiles del mes y lo multiplica por los días descontados.

Ejemplo: salario base de 660, mes con 22 días hábiles, dos faltas sin justificar. Cada día vale 660 ÷ 22 = 30, así que el descuento es 2 × 30 = 60.

Por qué `fn_DiasHabiles` no usa `DATEPART(WEEKDAY, ...)`. Esa función depende de la configuración del servidor (`DATEFIRST`) y del idioma, así que daría resultados distintos según dónde se instale. En cambio la función se apoya en aritmética: cualquier grupo de siete días seguidos tiene exactamente cinco hábiles, así que cuenta los días totales, multiplica las semanas completas por 5 y recorre los 0 a 6 días sueltos; para saber el día de la semana usa que el 1 de enero de 1900 fue lunes (`DATEDIFF(DAY, '19000101', fecha) % 7` da 0 si es lunes y `< 5` significa lunes a viernes). Limitación que se admite con naturalidad: no conoce los feriados.

Las dos funciones cortas valen por su nombre. `fn_IdBodega()` responde «¿cuál es el salón de la bodega?» en vez de escribir un `IdSalon = 1` fijo en todas partes. `fn_PeriodoCerrado(fecha)` responde «¿el mes de esta fecha está cerrado?»: los procedimientos de pagos y ausencias la consultan antes de tocar nada y, si devuelve 1, lanzan el error 50800 o 50801. Es la pieza que hace cumplir que **un mes cerrado es inmutable**.

`fn_AuditoriaCambios` abre los dos textos JSON (`Antes` y `Despues`) con `OPENJSON`, los alinea por nombre de campo con un `FULL OUTER JOIN` (porque en un alta no hay «antes» y en una baja no hay «después») y en una modificación deja solo los campos que difieren, como `SalarioBase: 600.00 → 650.00`. Es una vista con parámetros, y como trabaja con los nombres que vengan en el JSON, una columna nueva en una tabla se audita sin cambiar una línea.

Por qué función y no código repetido: la regla vive en un solo sitio; si cambia la forma de contar días hábiles, se corrige `fn_DiasHabiles` y todo lo que la usa queda corregido. **Límites que debes poder decir:** una función escalar se ejecuta una vez por fila (imperceptible con los cientos de empleados de una escuela, un problema con millones), y las vistas con `GETDATE()` dependen del reloj del servidor, que en una instalación local es el mismo equipo.

5. Procedimientos almacenados y transacciones: dónde viven las reglas

Un procedimiento almacenado es un programa guardado dentro de la base de datos que ejecuta una operación completa. Es la pieza donde viven las reglas del negocio y la que más te van a preguntar.

5.1 Por qué existen

La pantalla podría ejecutar un `INSERT` directo, pero entonces cada pantalla tendría que acordarse de todas las reglas. Con un procedimiento, la regla se escribe **una vez** y todas las pantallas que lo llaman la cumplen. Además corre dentro del servidor (valida y escribe en una sola ida y vuelta), puede protegerse con una transacción, recibe parámetros con tipo (si llega algo que no cabe en el tipo, falla antes de ejecutarse, lo que refuerza la defensa contra la inyección SQL) y lanza mensajes en español que la pantalla muestra tal cual.

La honestidad que te favorece: **no todo pasa por procedimientos**. Crear o editar un cargo, crear un consumible o dar de baja lógica a un empleado (pasarlo al estado 2) se hacen con `INSERT` o `UPDATE` directos desde la pantalla, siempre con parámetros. Siguen protegidos por las restricciones de la tabla y por los triggers (el historial se guarda igual), pero no tienen una regla de negocio compleja por encima. La regla que puedes defender: las operaciones con reglas serias o con más de una escritura (préstamos, bajas, pagos, planillas) pasan siempre por procedimientos.

5.2 Anatomía de uno: sp_RegistrarMovimientoConsumible

Es corto y tiene todas las partes. Léelo en este orden:

```
CREATE OR ALTER PROCEDURE dbo.sp_RegistrarMovimientoConsumible
    @IdConsumible INT, @Tipo VARCHAR(10), @Cantidad INT, @Motivo VARCHAR(40),
    @Destino VARCHAR(100) = NULL, @Usuario VARCHAR(50), @IdSalon INT = NULL
AS
BEGIN
    SET NOCOUNT ON; SET XACT_ABORT ON;
    BEGIN TRY
```

```
        IF @Tipo NOT IN ('Entrada', 'Salida')
            THROW 50040, 'El tipo de movimiento debe ser Entrada o Salida.',
1;
        IF @Cantidad IS NULL OR @Cantidad <= 0
            THROW 50041, 'La cantidad debe ser mayor que cero.', 1;

        BEGIN TRANSACTION;
        DECLARE @Stock INT;
        SELECT @Stock = StockActual FROM dbo.Consumibles WITH (UPDLOCK,
HOLDLOCK)
        WHERE IdConsumible = @IdConsumible;
        IF @Tipo = 'Salida' AND @Cantidad > @Stock
            THROW 50043, 'Stock insuficiente: no se puede despachar más de lo
que hay.', 1;

        DECLARE @Nuevo INT = CASE WHEN @Tipo = 'Entrada' THEN @Stock +
@Cantidad ELSE @Stock - @Cantidad END;
        UPDATE dbo.Consumibles SET StockActual = @Nuevo WHERE IdConsumible =
@IdConsumible;
        INSERT INTO dbo.MovimientosConsumible (IdConsumible, Tipo, Cantidad,
Motivo, Destino, SaldoResultante, Usuario, IdSalon)
        VALUES (@IdConsumible, @Tipo, @Cantidad, @Motivo, @Destino, @Nuevo,
@Usuario, @IdSalon);
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

El fragmento omite alguna validación del original, pero conserva la estructura:

1. **Parámetros (`@Algo`).** Lo que la pantalla le envía; los que tienen `= NULL` son opcionales.
2. **`SET NOCOUNT ON`.** Evita los mensajes de «N filas afectadas» que no sirven al programa.
3. **`SET XACT_ABORT ON`.** Si ocurre un error dentro de una transacción, la cancela entera (sin él, SQL Server a veces sigue con la siguiente instrucción y deja la transacción abierta a medias).
4. **Validaciones primero.** Cada regla incumplida lanza un `THROW` con un código propio y un mensaje en español. Todo lo anterior al `BEGIN TRANSACTION` no ha tocado datos.
5. **Transacción con bloqueo.** `UPDLOCK, HOLDLOCK` retiene la fila de stock mientras se calcula.

6. **Escritura doble.** Actualiza el stock y agrega el movimiento al kardex; ambos cambios se guardan juntos o ninguno.
7. **`CATCH`.** Si algo falla, pregunta si hay una transacción abierta (`@@TRANCOUNT > 0`), la deshace y termina con `THROW;` sin parámetros, que **vuelve a lanzar el mismo error** hacia C#. Si el `CATCH` solo deshiciera, el error se «tragaría» y la pantalla creería que todo salió bien.

El `CHECK (StockActual >= 0)` de la tabla es una segunda barrera: aunque el procedimiento tuviera un error, el stock nunca quedaría negativo.

5.3 Transacciones y bloqueos

Una **transacción** agrupa operaciones para que se cumplan todas o ninguna. La salida de 50 hojas exige dos escrituras (bajar el stock y anotar el kardex); si se corta la luz entre las dos, sin transacción el inventario y su historial se contradirían para siempre. Con `BEGIN TRANSACTION ... COMMIT` o `ROLLBACK`, esa mitad nunca queda guardada. Eso es la **A** de ACID (atomicidad); las otras tres: **C**onsistencia (se pasa de un estado válido a otro válido), a**I**slamiento (dos usuarios no ven ni pisan el trabajo a medias del otro) y **D**urabilidad (lo confirmado sobrevive aunque se apague el equipo, gracias al registro de transacciones). Otros casos: `sp_CerrarPeriodo` cierra el mes y escribe el historial; `sp_RegistrarMantenimiento` hace tres escrituras (el servicio, el estado de la unidad y el cierre del reporte de daño) y si el `THROW 50051` ocurre después de las dos primeras, el `CATCH` las deshace; `sp_GuardarConfiguracionInicial` guarda la configuración y crea el administrador en la misma transacción porque, si solo quedara la configuración, el asistente no volvería a aparecer y no habría ningún usuario con quien entrar.

¿Por qué algunos procedimientos no tienen `BEGIN TRANSACTION`? Una sola sentencia de `INSERT`, `UPDATE` o `DELETE` ya es atómica por sí misma. `sp_RegistrarPrestamo` solo tiene un `INSERT` al final, y `sp_CambiarEstadoUnidades` cambia veinte unidades con un único `UPDATE`. La transacción se pone donde protege algo y se evita donde sería ruido; y las validaciones que no necesitan datos bloqueados van antes, para mantenerla lo más corta posible.

`UPDLOCK` y `HOLDLOCK` resuelven la concurrencia. Hay 10 resmas y dos encargados registran a la vez una salida de 8: sin protección, los dos leen 10, ambos comprueban que 8 ⩽ 10 y ambos escriben; el stock termina en 2 y se entregaron 16 resmas (una *actualización perdida*). `UPDLOCK` avisa «la leo porque la voy a modificar» y `HOLDLOCK` mantiene el bloqueo hasta el final de la transacción; el segundo encargado espera, lee el stock real (2) y recibe el error 50043. El mismo patrón protege `sp_RegistrarDevolucion` (la segunda devolución del mismo préstamo recibe 50037), `sp_GuardarSalon` y `sp_CambiarEstadoUsuario`. No siempre hace falta: `sp_RegistrarPrestamo` usa un

`SELECT` normal, y si dos personas pidieran la misma unidad a la vez, el índice único filtrado `UX_Prestamos_UnidadActiva` rechazaría al segundo; la integridad nunca está en riesgo, solo cambia la calidad del mensaje.

5.4 THROW y los códigos de error

`THROW` detiene la ejecución y envía un error al programa: `THROW 50043, 'mensaje', 1;` lleva un número propio (mayor que 50000, para no chocar con los de SQL Server), el texto y un estado. El texto es lo que ve el usuario, por eso está escrito en español y pensado para él. Los números se agrupan por tema:

Rango	Tema	Ejemplo
50031 a 50037	Préstamos y devoluciones	50035: la unidad ya está prestada
50040 a 50043	Consumibles	50043: stock insuficiente
50200 a 50207	Carga académica	50204: supera 40 horas
50300 a 50311	Pagos	50306: ya existe un pago en ese período
50500 a 50514	Alta y asignación de unidades	50514: no hay tantas unidades disponibles
50800 a 50808	Cierre de mes	50800: el mes está cerrado
50900	Registro inmutable	El historial no se modifica ni se borra
50910 a 50914, 50962	Cambios de estado y bajas	50914: una baja no se revierte
50950 a 50953	Responsable de un préstamo	50952: el empleado no es docente

5.5 Los procedimientos que debes saber explicar

- **`sp_RegistrarPrestamo`.** Decide a quién se presta: Docente o Personal administrativo exigen un empleado real, activo y con cargo coherente; Alumno acepta texto libre. Valida la fecha límite futura, que la unidad exista, que no esté en mal estado, en reparación ni de baja, y que no tenga otro préstamo activo. Solo entonces inserta. El nombre del responsable lo toma del registro del empleado, no de lo que escribió el usuario.
- **`sp_RegistrarDevolucion`.** En una transacción y con `UPDLOCK` sobre el préstamo, lo cierra y cambia el estado de la unidad. Si vuelve como «Dado de baja» exige un motivo de al menos cinco caracteres (50962) y guarda fecha y motivo; una unidad ya dada de

baja no puede cambiar de estado (50914).

- **`sp_CambiarEstadoUnidades`.** Recibe una lista de números separados por coma (`'12,15,18'`), la convierte en filas con `STRING_SPLIT` y cambia todas con **un solo `UPDATE`**, que es más rápido que un bucle y atómico. Valida: no se da de baja una unidad prestada, no se marca «Bueno» una con reporte de daño abierto y una baja no se revierte.
- **`sp_AsignarUnidades`.** Pasa unidades de la bodega a un salón; cuenta las disponibles con bloqueo y rechaza con el 50514 si piden más de las que hay.
- **`sp_RegistrarPago`.** Revisa primero que el mes no esté cerrado (`fn_PeriodoCerrado`), luego montos, que el período no sea futuro, que no haya otro pago vigente del empleado en el mes y que un pago ya pagado no se edite.
- **`sp_GenerarPlanilla`.** Crea los pagos pendientes de todos los empleados que corresponden en un mes. Con `@Simular = 1` solo **cuenta** cuántos generaría, cuántos ya existen y cuántos empleados no tienen salario (es la vista previa de la pantalla); con `@Simular = 0` lo hace de verdad. Es un solo `INSERT ... SELECT` dentro de una transacción y es repetible: ejecutarlo dos veces no duplica pagos. El descuento de cada pago es el de su estructura más el de ausencias, limitado con un `CASE` para no superar base más bono y violar el `CHECK`. Rechaza meses futuros (50802) y cerrados (50800).
- **`sp_CerrarPeriodo` y `sp_ReabrirPeriodo`.** Cerrar exige que el mes tenga pagos y que ninguno siga pendiente, y escribe en `PeriodosPlanilla` y en el historial, todo en una transacción. Reabrir exige que esté cerrado y un motivo de al menos cinco caracteres; ambas acciones quedan en la auditoría.
- **`sp_AsignarDocente`.** Valida que el empleado sea docente, que la materia sea una de las suyas y que no se repita, y con bloqueo comprueba que el total de horas del docente en el año no pase de 40 (cuenta todas sus materias, no solo la de este grupo).
- **`sp_CambiarEstadoUsuario`.** Nadie puede desactivarse a sí mismo (50905) y debe quedar al menos otro administrador activo (50906). Si dos administradores se desactivaran mutuamente al mismo tiempo, ambas validaciones pasarían y el sistema quedaría sin administradores; por eso lee el usuario con `UPDLOCK, HOLDLOCK` dentro de una transacción.

5.6 Los 34 procedimientos, por familias

Familia	Procedimientos
Usuarios y acceso (8)	`sp_CrearUsuario`, `sp_ActualizarUsuario`, `sp_CambiarEstadoUsuario`, `sp_CambiarMiClave`, `sp_RestablecerClaveUsuario`, `sp_RegistrarAcceso`, `sp_RegistrarEvento`, `sp_GuardarConfiguracionInicial`
Personal y académico (3)	`sp_GuardarMateria`, `sp_GuardarMateriasEmpleado`, `sp_AsignarDocente`
Planillas y ausencias (9)	`sp_RegistrarPago`, `sp_MarcarPagado`, `sp_AnularPago`, `sp_GenerarPlanilla`, `sp_CerrarPeriodo`, `sp_ReabrirPeriodo`, `sp_RegistrarAusencia`, `sp_AnularAusencia`, `sp_RecalcularDescuentoAusencias`
Activos (10)	`sp_AltaUnidades`, `sp_AgregarUnidades`, `sp_AsignarUnidades`, `sp_MoverUnidades`, `sp_QuitarUnidades`, `sp_CambiarEstadoUnidades`, `sp_CambiarEstadoUnidad`, `sp_RegistrarPrestamo`, `sp_RegistrarDevolucion`, `sp_RegistrarMantenimiento`
Salones (2)	`sp_GuardarSalon`, `sp_CambiarEstadoSalon`
Consumibles (1)	`sp_RegistrarMovimientoConsumible`
Auditoría (1)	`sp_CrearTriggerAuditoria`

Una duplicación deliberada que debes saber defender: la regla de la baja de una unidad (no prestada, motivo de cinco caracteres, no se revierte) aparece en cuatro procedimientos porque hay cuatro caminos que pueden terminar en una baja (`sp_CambiarEstadoUnidades`, `sp_CambiarEstadoUnidad`, `sp_RegistrarMantenimiento` y `sp_RegistrarDevolucion`). Si estuviera en uno solo, los otros serían puertas traseras. El costo es de mantenimiento (cambiar el mínimo obliga a tocar cuatro sitios), y la mejora natural es un procedimiento auxiliar que las concentre.

6. Triggers y auditoría: la base que se vigila sola

Un trigger es código que la base ejecuta **automáticamente** cuando alguien inserta, modifica o borra filas de una tabla. Nadie lo llama: se dispara solo, y eso lo hace perfecto para el historial, porque nadie puede olvidarse de registrar un cambio. Con un procedimiento tú decides cuándo correr (`EXEC ...`); con un trigger, es el propio `INSERT`, `UPDATE` o `DELETE` el que lo despierta.

6.1 Cuándo se ejecuta y qué ve: `inserted` y `deleted`

Tus triggers de auditoría son `AFTER INSERT, UPDATE, DELETE`: se ejecutan después de que el cambio se aplicó, **dentro de la misma operación**. Dos consecuencias: si el trigger falla, el cambio completo se deshace (no puede haber un cambio sin su registro), y se ejecuta **una vez por sentencia**, no por fila: un `UPDATE` que cambia veinte empleados dispara el trigger una sola vez con las veinte filas, por eso se escribe con consultas sobre conjuntos. Existe otro tipo, `INSTEAD OF` («en lugar de»), donde el cambio original no se ejecuta y el trigger lo reemplaza; lo usa `trg_Auditoria_Inmutable`.

Dentro del trigger existen dos tablas temporales que el motor llena por ti:

Operación	`inserted` contiene	`deleted` contiene
`INSERT`	Las filas nuevas	Nada
`UPDATE`	Las filas con los valores nuevos	Las mismas filas con los valores viejos
`DELETE`	Nada	Las filas borradas

Con eso el trigger sabe qué pasó: filas en las dos es una modificación (`U`), solo en `inserted` un alta (`I`), solo en `deleted` una baja (`D`).

6.2 Quién hizo el cambio: `SESSION_CONTEXT`

El login de Windows no sirve para saber qué usuario actuó, porque todos los usuarios del programa se conectan con la misma cuenta del equipo (`Integrated Security`). La solución son dos piezas:

1. **En C#**, `Conexion.Conectar()` engancha el evento `StateChange`; cada vez que la conexión pasa de cerrada a abierta ejecuta `sp_set_session_context @key = 'Usuario', @value = <usuario de la sesión>`.
2. **En SQL**, el trigger lee ese valor con `SESSION_CONTEXT(N'Usuario')` y lo guarda como autor del cambio.

El valor vive solo mientras dure esa conexión. Así el registro dice «mgarcia modificó el pago» y no «el equipo de la oficina». Si no hay etiqueta (por ejemplo, alguien cambia datos desde Management Studio, o se usa el modo Debug del programa), el trigger guarda `NULL` y la vista lo muestra como `(sistema)`; el cambio igual queda registrado, solo falta saber quién. En cada fila también se guarda el nombre del equipo (`Equipo`, con `HOST_NAME()` por defecto). Una limitación honesta: el bloque de C# que pone la etiqueta tiene un `catch` vacío a propósito, para que un fallo de auditoría nunca impida trabajar.

6.3 El trigger de auditoría, paso a paso

Hay 17 triggers con el mismo molde, uno por tabla auditada (`trg_Aud_Empleados`, `trg_Aud_Prestamos`…). Este es el esqueleto:

```
CREATE OR ALTER TRIGGER dbo.trg_Aud_Empleados ON dbo.Empleados AFTER INSERT,
UPDATE, DELETE AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @op CHAR(1) = CASE WHEN EXISTS (SELECT 1 FROM inserted) AND
EXISTS (SELECT 1 FROM deleted) THEN 'U'
                               WHEN EXISTS (SELECT 1 FROM inserted) THEN 'I'
ELSE 'D' END;

    INSERT INTO dbo.Auditoria (Usuario, Tabla, Operacion, Clave, Antes,
Despues)
    SELECT CAST(SESSION_CONTEXT(N'Usuario') AS NVARCHAR(50)), N'Empleados',
@op,
           CAST(COALESCE(i.IdEmpleado, d.IdEmpleado) AS NVARCHAR(100)),
           x.Antes, x.Despues
    FROM inserted i FULL OUTER JOIN deleted d ON d.IdEmpleado = i.IdEmpleado
    CROSS APPLY (SELECT
        Antes   = CASE WHEN @op = 'I' THEN NULL ELSE (SELECT d.Nombre,
d.IdCargo /* ... */ FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END,
        Despues = CASE WHEN @op = 'D' THEN NULL ELSE (SELECT i.Nombre,
i.IdCargo /* ... */ FOR JSON PATH, WITHOUT_ARRAY_WRAPPER) END) x
    WHERE @op <> 'U' OR ISNULL(x.Antes, N'') <> ISNULL(x.Despues, N'');
END
```

En el original aparecen todas las columnas de `Empleados`. Las piezas importantes:

- **`@op`** decide `I`, `U` o `D` mirando qué tablas mágicas tienen filas.
- **`FULL OUTER JOIN`** une `inserted` y `deleted` por la llave; así una misma consulta sirve para los tres casos (en un alta falta el lado `deleted`, en una baja falta `inserted`).

- **`FOR JSON PATH`** convierte la fila en texto JSON (`{"Nombre":"Ana","IdCargo":3}`). Se guarda la fila **entera**, antes y después, porque es más simple y fiable; la pantalla luego muestra solo lo que cambió.
- **El `WHERE` final**: si fue una modificación pero el antes y el después son iguales (se guardó un formulario sin tocar nada), no se registra nada. Así el historial muestra cambios reales y no ruido.

El generador. Escribir 17 triggers a mano sería repetitivo y propenso a errores, así que `sp_CrearTriggerAuditoria` recibe el nombre de una tabla, descubre sus columnas y su llave primaria en el catálogo del sistema, arma el texto del trigger como cadena y lo ejecuta con `sp_executesql` (SQL dinámico, protegido con `QUOTENAME`). Omite las columnas binarias (fotos y logotipo) y acepta una lista de columnas a excluir: en `Usuarios` se excluyen `Clave` y `UltimoAcceso`, así que **ni siquiera el hash de las contraseñas se copia al registro**, y los inicios de sesión no generan un registro extra por el solo hecho de entrar.

6.4 Qué se guarda en `Auditoria`

Cada fila responde a seis preguntas: cuándo (`FechaHora`, hora del servidor), quién (`Usuario`), desde dónde (`Equipo`), en qué tabla (`Tabla`), qué se hizo (`Operacion`: `I`, `U`, `D` o `A` acceso) y sobre qué fila (`Clave`), más `Antes` y `Despues` en JSON. Ejemplo: un pago sube de 600 a 650; la fila tendría `Tabla = 'PagosPlanilla'`, `Operacion = 'U'`, `Clave = '41'` y el JSON completo antes y después. Es la vista `vw_Auditoria`, con `fn_AuditoriaCambios`, la que compara los dos textos y muestra solo `SalarioBase: 600.00 → 650.00`. El costo de guardar la fila completa es espacio en disco, insignificante para una escuela.

6.5 Un registro que no se puede alterar

Un historial que cualquiera puede editar no sirve como evidencia. `trg_Auditoria_Inmutable` es un trigger `INSTEAD OF UPDATE, DELETE` sobre `Auditoria`: cuando alguien intenta modificar o borrar una fila, se ejecuta **en lugar de** la operación y lanza el error 50900 («El registro de actividad no se puede modificar ni borrar»). Las altas sí están permitidas: es la forma en que se escribe el historial. **Qué tan seguro es:** protege contra errores y contra el uso indebido desde el programa o consultas normales, pero no es absoluto: una persona con permisos de administrador de la base podría desactivar el trigger o editar el archivo. Para una protección total harían falta permisos más estrictos, copias fuera del equipo o tablas de libro mayor (*ledger*). Presentarlo como una barrera real y razonable, no como invulnerable, demuestra que conoces el tema.

6.6 Historial del inventario y accesos

`trg_AuditoriaUnidades` y `trg_AuditoriaActivos` son más especializados: guardan en columnas normales (no JSON) el estado y el salón anterior y nuevo de cada unidad, para responder «¿dónde estuvo este pupitre y en qué estado?». Son selectivos (`trg_AuditoriaUnidades` registra solo si cambió el estado o el salón) y usan `ISNULL(SESSION_CONTEXT(N'Usuario'), SUSER_SNAME())`: sin usuario del programa, caen a la cuenta de Windows. No son redundantes: el registro de actividad es para quien audita todo el sistema y el historial de cada equipo es para quien consulta la vida de una pieza.

Además de los cambios en las tablas, el sistema deja constancia de **quién entra**: `sp_RegistrarAcceso` inserta en `Auditoria` una fila con `Tabla = 'Acceso'` y `Operacion = 'A'`, con el resultado en el JSON («Inició sesión», «Cerró sesión», «Intento fallido», «Intento con cuenta desactivada» o «Intento sin permiso»). Importa porque los intentos fallidos son la señal de alguien que intenta adivinar una contraseña y porque permite cruzar la hora de un cambio extraño con quién estaba dentro. La pantalla **Registro de actividad** lee `vw_Auditoria` y es de solo lectura.

7. Cómo se conecta la base con el programa en C#

La base de datos y el programa se hablan a través de una capa muy delgada. Entenderla te permite explicar qué pasa cuando alguien pulsa un botón.

7.1 Las capas y la conexión

El proyecto está dividido en dos: `Vistass` contiene las pantallas y `Modelos` contiene lo que no es pantalla (`Conexion`, `Datos`, `Sesion`, `ServicioUsuarios`, exportadores…). Las pantallas llaman a `Modelos` y `Modelos` habla con SQL Server. Las pantallas más nuevas piden a `Datos` que consulte; algunas antiguas abren la conexión con `Conexion.Conectar()` directamente, y las dos rutas usan la misma conexión, así que el usuario de la sesión se registra igual.

`Conexion.Conectar()` arma la cadena a **LocalDB** (`(localdb)\MSSQLLocalDB`, una versión liviana de SQL Server que no requiere instalar un servidor) con **seguridad integrada**: usa la cuenta de Windows, así que **no hay contraseña de base de datos en el código**. Además suscribe el evento que, cada vez que la conexión pasa de cerrada a abierta, ejecuta `sp_set_session_context` con el usuario que inició sesión: es la mitad C# de lo que viste en los triggers. Como está pensado para una sola computadora, usarlo en red exigiría una instancia de SQL Server y cambiar la constante `Servidor` de `Conexion` (y tener en cuenta que la autenticación de Windows en red exige que las cuentas existan en el servidor).

7.2 `Datos`: cuatro métodos para todo

Método	Para qué	Ejemplo de uso
`Tabla(sql, parámetros)`	Ejecuta una consulta y devuelve una tabla en memoria (`DataTable`)	Cargar la lista de préstamos desde `vw_Prestamos`
`Escalar(sql, parámetros)`	Devuelve un solo valor	El stock actual de un consumible
`Ejecutar(sql, parámetros)`	Ejecuta una instrucción sin devolver datos	Un `UPDATE` o `INSERT` sencillo
`Procedimiento(nombre, parámetros)`	Llama a un procedimiento almacenado	`sp_RegistrarPrestamo`

Cada llamada abre su conexión, ejecuta y la cierra gracias al `using`; no hay una conexión eterna abierta (no es lento porque ADO.NET recicla las conexiones físicas en un *pool*).

7.3 Parámetros: por qué nunca se pega texto al SQL

```
Datos.Procedimiento("sp_RegistrarMovimientoConsumible",
    Datos.P("@IdConsumible", id),
    Datos.P("@Tipo", cmbMovTipo.Text),
    Datos.P("@Cantidad", (int)numMovCant.Value),
    Datos.P("@Motivo", cmbMovMotivo.Text),
    Datos.PTexto("@Destino", destino),
    Datos.P("@Usuario", UsuarioActual()),
    Datos.P("@IdSalon", idSalon));
```

`Datos.P` crea un parámetro: los valores viajan **separados** del texto de la instrucción y el motor nunca los interpreta como código. Eso previene la **inyección SQL**: si alguien escribe `'; DROP TABLE Empleados; --` en un cuadro de texto, se guarda como texto, no se ejecuta. `Datos.PTexto` además convierte un texto vacío en `NULL`. Las listas de identificadores que llegan a un procedimiento se procesan con `STRING_SPLIT` y `CAST(value AS INT)`, así que un texto no numérico simplemente da error. Los filtros que ves en algunas pantallas (`"Tipo = '" + texto + "'"`) no son consultas al servidor sino `RowFilter` en memoria, así que no pueden alterar la base.

7.4 De la base a la tabla de la pantalla

Cuando abres Gestión de personal ocurre esta cadena: **(1)** la pantalla llama a `Datos.Tabla` con un `SELECT` sobre tablas o una vista; **(2)** SQL Server devuelve las filas y quedan en un `DataTable` en memoria; **(3)** se envuelve en un `DataView`, que filtra y ordena sin volver a la base (por eso el buscador responde al instante: modifica el `RowFilter`); **(4)** la tabla visual (`GridOscuro`) muestra el `DataView`. Las columnas de color vienen de la base: la alerta de préstamos y de consumibles se calcula en SQL y la pantalla solo traduce «Atrasado» o «Agotado» a un color. El filtro en memoria es lo adecuado para cientos o pocos miles de filas; con cientos de miles habría que filtrar en el servidor (otras pantallas, como Préstamos, ya mandan un `WHERE Alerta = 'Atrasado'` a la vista).

7.5 Qué pasa cuando algo falla

Si un procedimiento lanza un `THROW`, C# recibe una **`SqlException`**; la pantalla la atrapa y muestra su `Message`, que es el texto en español escrito en SQL. Así no hay que traducir códigos: la regla y su mensaje viven juntos en la base. Para las restricciones de las tablas el mensaje de SQL Server es técnico, y por eso en algunos puntos el programa reconoce el número: **2601 o 2627** (valor duplicado en un índice o restricción única) muestra «Ya existe un consumible con ese nombre» junto al campo, y **547** (llave foránea o `CHECK`) muestra «la estructura está en uso y no se puede eliminar». También puede buscar el nombre de la restricción en el mensaje (`UX_Empleados_DUI`, `UQ_Empleados_Nombre`), lo que funciona gracias a la convención de nombres.

7.6 Qué pantalla usa qué

Pantalla	Lee de	Escribe mediante	Se registra en el historial
Préstamos	`vw_Prestamos`	`sp_RegistrarPrestamo`, `sp_RegistrarDevolucion`	`trg_Aud_Prestamos`, `trg_AuditoriaUnidades`
Unidades de un equipo	`vw_UnidadesDetalle`	`sp_CambiarEstadoUnidades`, `sp_AltaUnidades`	`trg_Aud_UnidadesActivo`
Mantenimiento	`vw_Mantenimientos`	`sp_RegistrarMantenimiento`	`trg_Aud_MantenimientosActivo`
Consumibles	`vw_StockConsumibles`, `vw_KardexConsumibles`	`sp_RegistrarMovimientoConsumible`	El propio kardex
Asignar equipo a un salón	`vw_StockBodega`	`sp_AsignarUnidades`	`trg_AuditoriaUnidades`
Pagos de planilla	`vw_PagosPlanillaDetalle`	`sp_RegistrarPago`, `sp_MarcarPagado`, `sp_AnularPago`	`trg_Aud_PagosPlanilla`
Generar planilla y cerrar mes	La propia consulta de simulación	`sp_GenerarPlanilla`, `sp_CerrarPeriodo`, `sp_ReabrirPeriodo`	`trg_Aud_PeriodosPlanilla`

Pantalla	Lee de	Escribe mediante	Se registra en el historial
Ausencias	`vw_AusenciasDetalle`	`sp_RegistrarAusencia`, `sp_AnularAusencia`	`trg_Aud_Ausencias`
Carga académica	`vw_CargaDocente`	`sp_AsignarDocente`	`trg_Aud_AsignacionesDocente`
Registro de actividad	`vw_Auditoria`	Nadie: solo lectura	Es el historial mismo
Inicio de sesión	`Usuarios`	`sp_RegistrarAcceso`	`Auditoria` con operación A

7.7 Seguridad de cuentas y contraseñas

- **Contraseñas.** En `Usuarios.Clave` hay un hash BCrypt (`$2a$11$...`), nunca la contraseña: un hash es de un solo sentido, lleva una *sal* aleatoria (dos usuarios con la misma clave tienen hashes distintos) y es deliberadamente lento. El login lee el hash con una consulta parametrizada y `PasswordHasher.Verificar` compara **en C#**; la base jamás ve la contraseña en claro, que llega ya convertida en hash a `sp_CrearUsuario`, `sp_CambiarMiClave` y `sp_GuardarConfiguracionInicial`. El login acepta una clave antigua en texto plano (no empieza por `$2`) solo para migrarla: después de comprobar que la cuenta está activa y es administradora, la reemplaza por su hash.
- **Claves temporales.** Cuando un administrador crea otro usuario o restablece su contraseña, queda `DebeCambiarClave = 1` y el sistema obliga a cambiarla en el primer ingreso.
- **Recuperación por correo.** Se genera un código de seis dígitos con `RandomNumberGenerator` (el generador criptográfico, no `Random`), se envía por correo y se guarda solo en la memoria del formulario con vencimiento a los 15 minutos; son 5 intentos y luego se invalida. La contraseña nueva se guarda como hash con un `UPDATE` parametrizado desde C#. El mensaje «este correo no está registrado» revela qué correos existen: es un compromiso entre claridad y discreción que se presenta como mejora, no como defecto.
- **Contra la adivinación.** Después de cinco intentos fallidos con el mismo usuario el login se bloquea 60 segundos; las cuentas desactivadas no entran aunque tengan la clave correcta; cada intento rechazado queda auditado con su causa. Límite: el conteo vive en la memoria del programa (reiniciar lo borra) y es por usuario.
- **Quién entra.** `Roles` tiene Administrador y Empleado, pero el programa solo deja iniciar sesión a administradores; la tabla existe para que el modelo pueda crecer.
- **Presentar en Release.** En la configuración Debug el programa salta el login (y los cambios salen como `(sistema)`); para demostrar login, bloqueo y recuperación hay que presentar la compilación Release.

8. Tres flujos de punta a punta

Cuando domines estos tres recorridos podrás explicar cualquier otra parte del sistema, porque todas siguen el mismo patrón: **la pantalla pide, el procedimiento valida y escribe, las restricciones y los triggers vigilan y la vista devuelve el resultado.** Para cada flujo, termina siempre con las cuatro preguntas: qué problema resuelve, cómo, dónde se ve y qué pasa si falla.

8.1 Prestar un equipo y devolverlo

El préstamo

1. En la pestaña Préstamos eliges el equipo. La pantalla carga solo las unidades que se pueden prestar: se excluyen las de estado malo, en reparación o dadas de baja, y las que ya tienen un préstamo activo.
2. Eliges el tipo de responsable. Docente o Personal administrativo muestran una lista de empleados reales; Alumno permite escribir el nombre.
3. Al pulsar el botón, C# hace una primera revisión rápida (campos vacíos, fecha límite futura) y muestra los avisos junto a cada campo. **Esa es la primera capa, de comodidad.**
4. Si pasa, llama a `sp_RegistrarPrestamo` con la unidad, el tipo, el empleado, el aula, la fecha límite y el usuario que entrega.
5. **Segunda capa, la que garantiza:** el procedimiento repite las validaciones importantes en el servidor (empleado real, activo y con el cargo adecuado, errores 50950 a 50953; responsable no vacío, 50031; fecha futura, 50032; unidad existente, 50033; no en mal estado ni de baja, 50034; no prestada, 50035) e inserta en `Prestamos`.
6. **Tercera capa, la tabla:** el `INSERT` debe cumplir los `CHECK` y las llaves foráneas, y el índice único `UX_Prestamos_UnidadActiva` es la última red: si dos personas pidieran la misma unidad en el mismo instante, la segunda inserción fallaría.
7. El trigger `trg_Aud_Prestamos` se despierta solo y anota quién creó el préstamo, cuándo, desde qué equipo y cómo quedó la fila.
8. La pantalla recarga `vw_Prestamos`; la columna `Alerta` se calcula en ese momento («En curso», «Por vencer» o «Atrasado») y pinta la fila de rojo o ámbar.

La devolución

1. Seleccionas el préstamo, eliges el estado físico con que vuelve el equipo y pulsas Registrar devolución.
2. `sp_RegistrarDevolucion` abre una transacción y bloquea la fila del préstamo.
3. Verifica que siga activo (si no, 50037). Si el estado nuevo es «Dado de baja», exige un motivo de al menos cinco caracteres (50962).

4. Cierra el préstamo (`Estado = 'Devuelto'`, fecha de devolución) y actualiza `UnidadesActivo` con el estado nuevo; si entra en baja, guarda fecha y motivo.
5. Hace `COMMIT`; el trigger de unidades anota el cambio de estado en `AuditoriaUnidades`.

Si algo falla, el error sube por la misma ruta en sentido contrario: la base lanza `THROW` o una restricción, C# recibe una `SqlException` y la pantalla muestra el mensaje. Ejemplos: prestar una unidad ya prestada da el 50035; devolver una unidad dada de baja con otro estado, el 50914; devolver como «Dado de baja» sin motivo, el 50962.

8.2 Una salida de consumible

1. En Consumibles eliges el consumible y el tipo (Entrada o Salida). Si es salida, la pantalla consulta el stock actual y limita el numérico de cantidad a ese tope.
2. Según el motivo, el destino cambia: una lista de salones si es «Despacho a aula», texto obligatorio en los demás casos. Si es un salón, se envía también su número para enlazar el movimiento.
3. C# llama a `sp_RegistrarMovimientoConsumible`.
4. El procedimiento abre una transacción y bloquea la fila del consumible (`UPDLOCK, HOLDLOCK`). Si la salida pide más de lo que hay, lanza el 50043.
5. Calcula el saldo nuevo, actualiza `Consumibles.StockActual` y agrega una fila al kardex con el saldo resultante.
6. Hace `COMMIT`. El `CHECK (StockActual >= 0)` queda como segunda barrera.
7. La pantalla recarga `vw_StockConsumibles`; si el stock quedó en cero o bajo el mínimo, la columna `Alerta` lo marca.

La lección es la **defensa en capas**: el tope del numérico (comodidad), el procedimiento (regla) y el `CHECK` (red final) protegen lo mismo en tres niveles. Caso fallido: salida de 50 hojas cuando quedan 20 → el `THROW 50043` salta al `CATCH`, se hace `ROLLBACK`, `THROW;` relanza el error, C# lo recibe como `SqlException` y muestra «Stock insuficiente…»; ni el stock ni el kardex cambiaron.

8.3 El mes de planilla: de ausencias a cierre

1. **Se registra una ausencia.** `sp_RegistrarAusencia` valida (empleado activo, rango razonable, que incluya días hábiles, que el mes no esté cerrado, que no se cruce con otra) e inserta. En la misma transacción llama a `sp_RecalcularDescuentoAusencias`, que actualiza el descuento de los pagos pendientes de esos meses (no toca los pagados, los anulados ni los de meses cerrados).

2. **Se genera la planilla.** La pantalla llama primero a `sp_GenerarPlanilla` con `@Simular = 1` y muestra cuántos pagos se crearían y cuántos empleados no tienen salario asignado. Al confirmar, la llama con `@Simular = 0`: crea un pago pendiente por empleado activo con el salario de su estructura y el descuento por ausencias calculado con las funciones de la sección 4.
3. **Se revisan y se pagan.** Un pago pendiente se edita con `sp_RegistrarPago`; `sp_MarcarPagado` lo pasa a pagado. Un pago pagado ya no se edita: se anula (`sp_AnularPago`) y se registra de nuevo; el índice filtrado lo permite porque ignora los anulados.
4. **Se cierra el mes.** `sp_CerrarPeriodo` exige que el mes tenga pagos y que ninguno siga pendiente; escribe en `PeriodosPlanilla` y en el historial.
5. **Después del cierre.** Cualquier intento de crear o modificar pagos o ausencias de ese mes choca con `fn_PeriodoCerrado` y recibe el error 50800 o 50801. Solo un administrador puede reabrir el mes con `sp_ReabrirPeriodo`, y también queda en el historial.

Este flujo muestra bien por qué la lógica está en la base: ocho procedimientos, cuatro funciones y cuatro tablas cooperan, y la pantalla no necesita conocer ninguna de esas reglas. Si se va la luz mientras se genera la planilla, la base queda como estaba antes de empezar: la generación es una sola transacción y, al reiniciar, SQL Server recupera el estado confirmado desde su registro de transacciones; se puede volver a generar sin duplicar pagos.

9. Cómo leer un script de SQL Server: `GO`, `SET`, `dbo` y `CREATE TABLE`

Esta sección es para abrir `BaseDeDatos_Completa.sql`, mirar cualquier bloque y saber qué está diciendo. Usa como ejemplo la tabla `Empleados`, porque reúne casi todos los elementos del resto del script, y empieza por la cabecera que la precede. En el archivo real, `GO` está **solo en su propia línea**.

9.1 ¿Qué es un script y qué significa `GO`?

Un script es un archivo de texto con instrucciones SQL que se ejecutan de arriba hacia abajo. SQL Server no lo recibe entero sino en **lotes** (*batches*), y `GO` marca dónde termina un lote y empieza el siguiente. Dato clave: **`GO` no es SQL**; no lo entiende el servidor sino la herramienta (Management Studio o `sqlcmd`), que corta el archivo en esos puntos y envía cada pedazo por separado. Importa por tres razones: `CREATE VIEW`, `CREATE PROCEDURE`, `CREATE FUNCTION` y `CREATE TRIGGER` deben ser **lo primero de su lote** (por eso cada una termina con su `GO`); las variables de `DECLARE` solo viven dentro de su lote; y algunos

cambios (como `USE`, que cambia la base de trabajo) deben quedar completos antes de que las líneas siguientes los aprovechen. `GO 5` repetiría el lote cinco veces; no se usa en este proyecto.

9.2 La cabecera: las opciones `SET`

```
SET NOEXEC OFF;
GO
SET QUOTED_IDENTIFIER ON;
SET ANSI_NULLS ON;
SET ANSI_PADDING ON;
SET ANSI_WARNINGS ON;
SET ARITHABORT ON;
SET CONCAT_NULL_YIELDS_NULL ON;
SET NUMERIC_ROUNDABORT OFF;
SET NOCOUNT ON;
GO
```

Un `SET` cambia una **regla de comportamiento de la sesión actual** (esa conexión mientras esté abierta). Estas se fijan al inicio para que el script dé el mismo resultado sin importar con qué herramienta se ejecute, porque cada herramienta trae valores iniciales distintos.

Opción	Qué significa estando así	Ejemplo en este proyecto
`QUOTED_IDENTIFIER ON`	Las comillas dobles nombran objetos y los textos van con comillas simples	**Obligatoria** para los índices filtrados como `UX_Empleados_DUI`
`ANSI_NULLS ON`	Comparar con `NULL` usando `=` nunca da verdadero: hay que escribir `IS NULL`	`WHERE DUI IS NOT NULL` en ese índice
`ANSI_PADDING ON`	Un `VARCHAR` guarda los espacios finales tal como llegan	Los textos se almacenan fielmente
`ANSI_WARNINGS ON`	Un texto demasiado largo produce un **error** en lugar de cortarse sin avisar	Un nombre de más de 200 caracteres se rechaza

Opción	Qué significa estando así	Ejemplo en este proyecto
`ARITHABORT ON`	Si una operación aritmética se desborda o divide por cero, la consulta se cancela	Evita resultados numéricos inválidos en silencio
`CONCAT_NULL_YIELDS_NULL ON`	Sumar un texto con `NULL` da `NULL`	`e.Nombre + ISNULL(' · ' + c.NombreCargo, '')`: sin cargo, el trozo se vuelve `NULL` y `ISNULL` lo cambia por vacío
`NUMERIC_ROUNDABORT OFF`	Perder precisión al redondear **no** lanza error	Los cálculos de salarios con `DECIMAL`
`NOCOUNT ON`	No devuelve «(N filas afectadas)» tras cada instrucción	Reduce ruido

¿Por qué precisamente esas siete? Porque SQL Server exige esa combinación (seis en `ON` y `NUMERIC_ROUNDABORT` en `OFF`) para trabajar con **índices filtrados**; si alguien ejecutara el script con otra, las inserciones podrían fallar con un mensaje que parece no tener relación. En una frase de defensa: «la cabecera fija las reglas estándar para que la base se cree igual en cualquier equipo, y es indispensable para los índices filtrados».

9.3 La guardia del script: `SET NOEXEC`

`NOEXEC ON` significa «compila todo pero **no ejecutes nada**». Después de crear la base y cambiar a ella con `USE`, el script comprueba que de verdad estamos dentro:

```
IF DB_NAME() <> N'Gestion_administrativa_y_escolar'
BEGIN
    PRINT N'*** No se pudo crear o abrir la base... ***';
    SET NOEXEC ON;
END
GO
```

Si el `USE` falló, Management Studio seguiría ejecutando los lotes siguientes **en otra base**, normalmente `master`, y crearía 30 tablas donde no corresponde. Con `NOEXEC ON` el resto se compila pero no se ejecuta. Al final del archivo hay otro `SET NOEXEC OFF;` y el del inicio sirve por si una ejecución cancelada la dejó encendida. En esa zona están también `IF DB_ID(N'...') IS NULL CREATE DATABASE ...;` (crea la base solo si no existe) y `USE ...;`.

9.4 `IF OBJECT_ID(N'dbo.Empleados', 'U') IS NULL`

Se lee de adentro hacia afuera: `OBJECT_ID(nombre, tipo)` devuelve el número interno de un objeto de la base, o `NULL` si no existe. La `N` antes de las comillas indica texto Unicode (solo es formato). **`dbo.`** es el **esquema**, algo así como la carpeta o el dueño lógico del objeto: `dbo` (*database owner*) es el esquema por defecto y el que usa todo el proyecto, por eso cada tabla se llama `dbo.Algo`. **`'U'`** es el tipo de objeto: tabla de usuario (`V` vista, `P` procedimiento, `TR` trigger, `FN` función). Entonces la línea dice: «si la tabla `Empleados` no existe, créala». Un `IF` sin `BEGIN ... END` controla solo la instrucción siguiente, aquí el `CREATE TABLE` completo. Así el script se puede ejecutar de nuevo sin destruir nada: es la **idempotencia**. Las vistas, funciones, procedimientos y triggers usan `CREATE OR ALTER`, que los crea o los actualiza sin fallar, y los datos iniciales se insertan solo si faltan.

9.5 El `CREATE TABLE`: columnas y restricciones

Cada línea de columna sigue el patrón **nombre → tipo → obligatoriedad → extras**. `Nombre VARCHAR(200) NOT NULL` se lee «la columna Nombre es un texto de hasta 200 caracteres y es obligatoria».

Tipo	Qué guarda	Ejemplo
`INT`	Entero, hasta unos 2.147 millones	`IdEmpleado`, las llaves
`VARCHAR(n)`	Texto de hasta *n* caracteres; ocupa solo lo que se escribe	`Nombre VARCHAR(200)`
`DATE`	Una fecha, sin hora	`FechaContratacion`
`DATETIME`	Fecha con hora	`FechaLimite` de los préstamos
`BIT`	Verdadero o falso (1 o 0)	`Estado` de `Planillas`, `Activo`, `Anulado`
`DECIMAL(10,2)`	Decimales exactos: 10 dígitos en total, 2 después de la coma	`SalarioBase`, máximo 99.999.999,99

Las fotos son `VARBINARY(MAX)` en una tabla aparte (`EmpleadoFotos`) para que las consultas de empleados no arrastren imágenes. El DUI y el teléfono van como texto aunque tengan dígitos: no se suman, pueden empezar con cero y el DUI lleva un guion. `NVARCHAR` guarda Unicode completo al doble de espacio; para español con tildes y eñes alcanza `VARCHAR`. En `Empleados` solo `Nombre` es obligatorio: `IdCargo`, `IdEstado` e `IdPlanilla` (los números que apuntan a `Cargos`, `EstadosEmpleado` y `Planillas`) aceptan `NULL`, y que tengan valor lo exige el formulario.

Al final del `CREATE TABLE` vienen las **restricciones**, cada una con `CONSTRAINT <nombre>` y su tipo de regla; léelas en voz alta:

- `CONSTRAINT PK_Empleados PRIMARY KEY (IdEmpleado)`: «IdEmpleado identifica cada fila: no puede repetirse ni estar vacío» (y, por defecto, ordena físicamente la tabla por esa columna, lo que acelera la búsqueda por número).
- `CONSTRAINT UQ_Empleados_Nombre UNIQUE (Nombre)`: «no hay dos empleados con el mismo nombre». Límite: dos personas reales con el mismo nombre; el identificador verdadero es el DUI, por eso existe `UX_Empleados_DUI`.
- `CONSTRAINT FK_Empleados_Cargo FOREIGN KEY (IdCargo) REFERENCES dbo.Cargos(IdCargo)`: «el valor de `IdCargo` debe existir en `Cargos`, o estar vacío»; no se puede borrar un cargo en uso (error 547).
- `CONSTRAINT CK_Empleados_DUI CHECK (...)`: «la fila solo se acepta si esta condición se cumple».

El `CHECK` del DUI es `CHECK ([DUI] IS NULL OR [DUI] like '[0-9][0-9][0-9][0-9][0-9][0-9][0-9][0-9]-[0-9]')`. Los corchetes alrededor de `[DUI]` solo son una forma de escribir el nombre. La condición une dos partes con `OR`: o está vacío, o cumple el patrón; `[0-9]` es «un carácter entre 0 y 9», escrito ocho veces, un guion y otro `[0-9]`. Acepta `01234567-8` y rechaza `1234567-89`, `0123456789` y `0123456a-8`. El `IS NULL OR` es redundante (un `CHECK` con `NULL` pasa de todos modos) y está para dejar la intención explícita. **Este `CHECK` valida solo el formato; el dígito verificador se calcula en el programa, en C#, no en la base.**

Una tabla no termina en su `CREATE TABLE`: justo después el script agrega con instrucciones aparte el índice `UX_Empleados_DUI`, y más abajo el trigger `trg_Aud_Empleados` y la vista `vw_EmpleadosDetalle`. Para entender una tabla completa, busca su nombre en todo el archivo con Ctrl+F.

9.6 Receta para leer cualquier `CREATE TABLE` y un ejercicio

1. Busca `IF OBJECT_ID(...) IS NULL`: te dice qué tabla es y confirma que es segura de volver a ejecutar.
2. Lee las columnas de arriba abajo: nombre → tipo → `NOT NULL` / `NULL` → extras (`IDENTITY`, `DEFAULT`).
3. Identifica la llave primaria (`PK_`).
4. Lee las llaves foráneas (`FK_`): son las flechas que dibuja el diagrama.
5. Lee los `UNIQUE` y `CHECK`: son las reglas de negocio que la tabla hace cumplir por sí sola.

Pruébalo con esta tabla del mismo script:

```
CREATE TABLE dbo.Planillas (
    IdPlanilla  INT IDENTITY(1,1) NOT NULL,
    SalarioBase DECIMAL(10,2) NOT NULL,
    Descuento   DECIMAL(10,2) NOT NULL CONSTRAINT DF_Planillas_Descuento
DEFAULT 0,
    Bono        DECIMAL(10,2) NOT NULL CONSTRAINT DF_Planillas_Bono DEFAULT
0,
    SalarioNeto DECIMAL(10,2) NOT NULL,
    Estado      BIT NOT NULL CONSTRAINT DF_Planillas_Estado DEFAULT 1,
    CONSTRAINT PK_Planillas PRIMARY KEY (IdPlanilla)
);
```

¿Qué columnas son obligatorias? ¿Qué pasa si inserto una planilla sin indicar el bono? ¿Cuál es el salario máximo que cabe? ¿Hay llaves foráneas? **Respuestas:** todas son obligatorias (ninguna acepta `NULL`); sin bono se guarda `0` por el `DEFAULT 0` (igual con el descuento y con `Estado`, que vale 1); el máximo es 99.999.999,99; y no hay llaves foráneas: esta tabla no apunta a ninguna, son otras (como `Empleados`) las que apuntan a ella.

Comprobación rápida. ¿Para qué sirve `GO`? Para separar el script en lotes; no es SQL sino una orden de la herramienta. ¿Por qué tantas líneas `SET`? Para fijar las reglas estándar y poder crear índices filtrados sea cual sea la herramienta. ¿Qué evita `NOEXEC`? Que, si la base no se pudo crear o abrir, el resto del script se ejecute en otra (normalmente `master`). ¿Qué distingue `PRIMARY KEY`, `UNIQUE` y `FOREIGN KEY`? La primera identifica cada fila, la segunda impide repetidos en otra columna y la tercera obliga a que el valor exista en otra tabla. ¿Por qué se nombran las restricciones? Porque su nombre aparece en los errores y el programa los usa para avisar al usuario con precisión.

10. Preguntas y respuestas esenciales

Son las 40 preguntas que más pesan de las 73 de la guía completa (secciones 10 a 15), con cada respuesta reducida a lo que necesitas decir en 30 a 60 segundos. Las marcadas con ★ son las que casi seguro caen: empieza por ellas. Cómo estudiarlas: lee la pregunta, tapa la respuesta, dila con tus palabras y compara. Si te trabas, ese punto es el que debes releer; al final de cada respuesta va el número de la pregunta en la guía completa por si quieres el detalle.

10.1 Conceptos y diseño

P1 ★. ¿Qué es una base de datos relacional y por qué no guardar todo en archivos? ¿Cómo está organizada la suya?

Organiza la información en tablas relacionadas por llaves: cada tabla trata de un tema y, cuando necesita referirse a otro dato, guarda su número en vez de copiarlo (el cargo «Director» existe una sola vez). Con archivos u hojas de cálculo hay tres problemas: datos repetidos que se contradicen, nada impide valores absurdos (un pago negativo) y dos personas se pisan el trabajo; SQL Server los resuelve con relaciones, restricciones y transacciones con bloqueos. Se eligió porque el sistema es C# con Windows Forms y LocalDB viene con Visual Studio. La base tiene 30 tablas en siete grupos que coinciden con el menú, más 27 vistas, 34 procedimientos, 6 funciones y 20 triggers: las tablas guardan, las vistas muestran, los procedimientos deciden, las funciones calculan y los triggers vigilan. *(Completa: 10.1, 10.2)*

P2 ★. ¿Por qué separaron el catálogo de equipos de las unidades físicas?

Porque «Pupitre» y «este pupitre» no son lo mismo. `GestionActivos` es el catálogo (qué tipos existen) y `UnidadesActivo` es la pieza concreta, con su código, estado, salón, costo y proveedor. Al principio el estado vivía en el catálogo, y si un pupitre se rompía el sistema consideraba que todos lo estaban: era imposible decir «este está en el aula 3 y dañado, aquel está nuevo en bodega». Al llevar el estado a la unidad, cada pieza se presta, se da de baja y se audita por separado. Es normalización: la información vive en el nivel al que realmente pertenece. *(10.3)*

P3. ¿Qué es la integridad de los datos y cómo la garantizan?

Que los datos guardados sean coherentes y cumplan las reglas. Hay tres tipos: la de **entidad** (cada fila se identifica de forma única: llaves primarias), la **referencial** (las referencias apuntan a algo que existe: llaves foráneas) y la de **dominio** (cada valor es razonable: `CHECK`, tipos de dato y `DEFAULT`). Todo eso lo aplica el motor siempre, venga el dato de donde venga; encima se añaden las reglas elaboradas de los procedimientos y triggers, y la pantalla ayuda al usuario a no equivocarse. *(10.4)*

P4 ★. ¿Por qué pusieron la lógica en la base y no solo en el programa?

Porque el programa es solo una de las formas de llegar a los datos. Si las reglas viven en las pantallas, cada pantalla nueva tiene que recordarlas todas y un olvido deja pasar datos incorrectos; en la base se escriben una vez y se cumplen siempre. Ejemplo: la regla de baja (motivo de al menos cinco caracteres) está en cuatro procedimientos porque hay cuatro caminos que pueden terminar en una baja; si estuviera solo en una pantalla, los otros tres se la saltarían. Además la pantalla puede rediseñarse por completo sin tocar las reglas. No todo está en la base: las validaciones rápidas también están en la pantalla para avisar junto al campo. **La pantalla da comodidad; la base da garantía.** *(10.5)*

P5. ¿Cómo se instala la base, qué pasa si ejecutan el script dos veces y cómo funciona el primer uso?

Se ejecuta `BaseDeDatos_Completa.sql` completo en Management Studio. Repetirlo no rompe nada (**idempotencia**): cada tabla se crea solo si no existe, vistas, funciones, procedimientos y triggers usan `CREATE OR ALTER` y los datos se insertan solo si faltan; además una guardia (`NOEXEC`) detiene el script si la base no se pudo abrir. El script **no crea usuarios**: al abrir el programa por primera vez, si `ConfiguracionEmpresa` está vacía, un asistente pide los datos de la institución y del primer administrador (con el correo verificado por código); el programa calcula el hash BCrypt y llama a `sp_GuardarConfiguracionInicial` , que guarda todo en una transacción. Si ya hay configuración lanza el 50021, y un `CHECK` obliga a que la única fila tenga `IdConfig = 1` . *(10.6, 10.8)*

P6 ★. ¿Qué diferencia hay entre dar de baja y eliminar? ¿Cómo cubre el sistema el CRUD?

Eliminar destruye la fila; dar de baja la marca y conserva el historial. Casi todo tiene historia (un empleado que salió sigue en los pagos que recibió), por eso el empleado pasa al estado 2, los pagos y ausencias se anulan con una bandera (`Anulado` , `Anulada`) que los índices ignoran, la unidad pasa a «Dado de baja» con fecha y motivo, y el usuario y el salón se desactivan. Se borra físicamente lo que no tiene historia propia (la foto, una carga asignada por error, una estructura salarial sin uso, un activo sin unidades), siempre con la doble defensa pantalla más llave foránea. En Personal, `frmAgregarPersonal` hace `INSERT` o `UPDATE` con parámetros, la lista se lee con `JOIN` y filtros, y eliminar es una baja lógica; los módulos con reglas (préstamos, consumibles, planillas) crean y modifican por procedimientos. *(10.7, 15.2)*

10.2 Tablas, llaves y restricciones

P7 ★. ¿Qué diferencia hay entre llave primaria y foránea? ¿Puede una columna ser ambas?

La primaria responde «¿qué fila eres?»: es única y no vacía (`Empleados.IdEmpleado`). La foránea responde «¿de qué otra fila dependes?»: `Empleados.IdCargo` debe existir en `Cargos` . El motor lo impide en las dos direcciones (no asignar un cargo inexistente ni borrar uno en uso, que dejaría una fila huérfana), aunque alguien escriba SQL directo. Una columna puede ser ambas porque hablan de cosas distintas, unicidad y existencia: `EmpleadoFotos.IdEmpleado` es primaria (una foto por empleado) y foránea hacia `Empleados` ; en `EmpleadoMaterias` la primaria es la pareja `(IdEmpleado, IdMateria)` y cada una es además foránea. *(11.1)*

P8. ¿Para qué sirve `IDENTITY` y por qué `Salones` no lo usa?

`IDENTITY(1,1)` hace que el motor numere solo (1, 2, 3…), de forma segura aunque dos personas guarden a la vez; el número no se reutiliza (puede haber huecos) y no se puede insertar uno propio sin `SET IDENTITY_INSERT` . `Salones.IdSalon` no lo tiene:

`sp_GuardarSalon` calcula `ISNULL(MAX(IdSalon), 0) + 1` a mano, y para que dos administradores no obtengan el mismo número lee el máximo con `UPDLOCK, HOLDLOCK` dentro de una transacción. No se defiende como mejor que `IDENTITY` (que es más simple y rápido): es un rasgo conservado y compensado, y cambiarlo exigiría recrear la tabla y sus relaciones. *(11.2)*

P9 ★. ¿Qué es un índice único filtrado y por qué no un `UNIQUE` normal?

Es un `UNIQUE` con `WHERE`: «no hay dos filas con el mismo valor, contando solo las que cumplan el filtro». Sirve para reglas del tipo «solo una mientras esté vigente». `UX_Empleados_DUI` (`WHERE DUI IS NOT NULL`): el DUI es opcional y un `UNIQUE` normal trata los `NULL` como iguales, así que permitiría un solo empleado sin DUI. `UX_Prestamos_UnidadActiva` (`Estado = 'Activo'`): una unidad no se presta dos veces a la vez, pero puede tener muchos préstamos ya devueltos; además es la red si dos personas pulsan al mismo tiempo. `UX_PagosPlanilla_Empleado_Periodo` (`Anulado = 0`): un pago vigente por empleado y mes; al anularlo deja de contar y se puede registrar otro. Se viola con el error 2601 (los índices) o 2627 (un `UNIQUE`). *(11.3)*

P10 ★. ¿Diferencia entre un `CHECK`, un trigger y una validación en C#?

Un `CHECK` mira solo la fila que se guarda (`CK_Salones_Capacidad`: entre 1 y 500; `CK_PagosPlanilla_Montos` compara columnas de la misma fila). Es instantáneo y no se puede esquivar, pero no ve otras filas ni tablas, y con `NULL` la fila pasa (para exigir un dato hay que usar `NOT NULL`). Cuando la regla necesita otras filas la valida un procedimiento: el tope de 40 horas semanales de un docente suma varias filas (`sp_AsignarDocente`, error 50204). Un trigger no valida entradas: reacciona a cambios ya hechos (auditar, impedir ediciones). La validación en C# es comodidad, no seguridad: se la salta quien llegue a la base sin pasar por el formulario. Regla: **lo que debe ser cierto siempre va en la base; lo que ayuda al usuario va además en la pantalla.** *(11.4)*

P11. ¿Qué hace `ON DELETE CASCADE` y por qué solo está en cinco relaciones?

Al borrar el padre, el motor borra solo las hijas. Está en `EmpleadoFotos`, `EmpleadoMaterias` y `AsignacionesDocente` (respecto a `Empleados`) y en `UnidadesActivo` y `ReportesDanio` (respecto a `GestionActivos`): filas que no tienen sentido sin su padre. Lo revelador es dónde no está: `PagosPlanilla` y `Prestamos` son historial, así que un empleado con pagos no se puede borrar físicamente, y por eso se da de baja. La cascada evalúa toda la cadena: si una fila queda protegida por otra llave sin cascada, se cancela el borrado completo. *(11.5)*

P12. ¿Qué es `SalarioNeto` como columna calculada? ¿Por qué existen `Planillas` y `PagosPlanilla`?

`SalarioNeto AS (SalarioBase + Bono - Descuento) PERSISTED`: nadie la escribe, el motor la calcula y la guarda, así que nunca se contradice con sus componentes (y se corrige sola cuando se recalcula el descuento por ausencias). `Planillas` es el catálogo de estructuras salariales; `PagosPlanilla` es lo que realmente se pagó, una fila por empleado y mes. Copiar los montos al pago es intencional: es una fotografía de ese mes (como una factura con el precio del día), para que subir el salario en abril no cambie el pago de marzo. El pago tiene sus reglas: período siempre día 1, estado Pagado o Pendiente, montos coherentes y un único pago vigente. Diferencia honesta: el `SalarioNeto` de `Planillas` es una columna normal; hacerla calculada sería una mejora coherente. *(11.6, 11.7)*

P13. ¿Por qué escogieron esos tipos de dato?

`DECIMAL(10,2)` para el dinero porque es exacto (con `FLOAT`, 0.1 + 0.2 puede dar 0.30000000000000004 y en una planilla los centavos no cuadran). El DUI y el teléfono van como `VARCHAR`: no se suman, pueden empezar con cero y el DUI lleva guion. `DATE` donde no hay hora (nacimientos, períodos) y `DATETIME` donde sí (préstamos, auditoría). `BIT` para verdadero o falso. Las fotos en `VARBINARY(MAX)` y en una tabla aparte. `VARCHAR` alcanza para el español; unas pocas columnas de `Usuarios` usan `NVARCHAR`. Los largos máximos distintos acotan lo que cabe. *(11.8)*

P14. ¿Cómo se representa que un docente da varias materias y una materia la dan varios docentes?

Es una relación muchos a muchos y se resuelve con una tabla intermedia: `EmpleadoMaterias` (`IdEmpleado`, `IdMateria`), con llave primaria compuesta que impide repetir la pareja y cada columna foránea. No la confundas con `AsignacionesDocente`: `EmpleadoMaterias` es lo que el docente **puede** dar (su habilitación) y `AsignacionesDocente` es lo que **da** (sección, año lectivo y horas entre 1 y 40, única por `(IdEmpleado, IdSeccion, IdMateria, AnioLectivo)`). `sp_AsignarDocente` consulta las dos. Pista general: una tabla cuya llave primaria es una pareja de llaves foráneas casi siempre representa un muchos a muchos. *(11.9)*

P15. ¿Qué pasa si el usuario intenta borrar algo que otra tabla está usando? ¿Quién lo detiene?

Hay dos defensas. La **amable**: en el catálogo de salarios la pantalla cuenta cuántos empleados usan la estructura (incluidos los dados de baja) y avisa. La **dura**: si eso se saltara (otra persona la asignó un segundo antes), el `DELETE` choca con `FK_Empleados_Planilla`, el motor lo cancela con el error 547 y C# lo traduce a «está en uso y no se puede eliminar». La integridad no depende de que el programador no se equivoque: la pantalla se ocupa de la experiencia y la base de la verdad; y la baja lógica y la llave foránea trabajan juntas. *(11.10)*

10.3 Vistas y funciones

P16 ★. ¿Qué es una vista? ¿Por qué tantas? ¿Qué diferencia hay con una función y un procedimiento?

Una vista es una consulta guardada con nombre: no guarda datos (siempre está al día) y no es más rápida; su valor es la organización. Si cada pantalla escribiera sus `JOIN`, quedarían copiados en diez sitios (`vw_UnidadesDetalle` une cinco tablas); la vista además traduce números a palabras y calcula columnas una sola vez. Las tablas se organizan para guardar sin repetir, las vistas para leer cómodamente. **Vista:** consulta fija, sin parámetros. **Función:** recibe parámetros, devuelve un valor o una tabla, se usa dentro de otra consulta y no puede modificar datos. **Procedimiento:** se invoca con `EXEC`, puede abrir transacciones, modificar datos y lanzar errores. `fn_AuditoriaCambios` es una «vista con parámetros». *(12.1, 12.2, 12.7)*

P17. ¿Por qué algunas vistas usan `JOIN` y otras `LEFT JOIN`?

La diferencia es qué pasa con las filas sin pareja: `JOIN` las descarta y `LEFT JOIN` las conserva con `NULL`. La pregunta de decisión es «¿puede existir esta fila sin la otra?». En `vw_UnidadesDetalle` el equipo, el salón y el estado son obligatorios (`JOIN`) y la categoría puede ser nula (`LEFT JOIN`); en `vw_EmpleadosDetalle` el cargo, la estructura y el pago del mes pueden faltar (`LEFT JOIN`), y el pago del mes usa `OUTER APPLY` con `TOP 1`. Al contar con un `LEFT JOIN` se usa `COUNT(u.IdUnidad)` y no `COUNT(*)`, para que un salón sin unidades dé 0. *(12.3)*

P18 ★. ¿Cómo funcionan las alertas de `vw_Prestamos` y `vw_StockConsumibles`? ¿Y `vw_StockBodega`?

Son columnas calculadas con `CASE`, que se evalúa de arriba abajo (por eso `Devuelto` se pregunta primero). Con `GETDATE()` la alerta cambia sola con el tiempo: «En curso», «Por vencer» (falta menos de dos días) y «Atrasado», sin actualizar ninguna fila. La pantalla filtra, ordena, cuenta y pinta por esa columna, y si el negocio decide que «por vencer» es a tres días, se cambia un número en la vista. `vw_StockBodega` cuenta las unidades en la bodega (`fn_IdBodega()`), sin estado problemático y con un `NOT EXISTS` de préstamo activo: «Disponibles» es exactamente lo asignable. *(12.4, 12.5)*

P19. ¿Cómo cuenta `fn_DiasHabiles` los días y cómo se calcula el descuento por ausencias?

Con aritmética: siete días seguidos tienen cinco hábiles, así que multiplica las semanas completas por 5 y recorre los 0 a 6 días sueltos; el día de la semana sale de que el 1 de enero de 1900 fue lunes (`DATEDIFF(DAY, '19000101', f) % 7`, y `< 5` es lunes a viernes). No usa `DATEPART(WEEKDAY, ...)` porque depende de `DATEFIRST` y del idioma del servidor; tampoco conoce los feriados. El descuento: `fn_DiasDescontados` cuenta las faltas injustificadas y los permisos sin goce recortados al mes, y `fn_DescuentoAusencias`

aplica salario ÷ días hábiles × días descontados: con 660 y 22 días, cada día vale 30 y dos faltas descuentan 60. `sp_GenerarPlanilla` lo suma al descuento de ley, limitado a base más bono. *(12.8, 12.9)*

P20. ¿Cómo se construye el registro de actividad legible a partir de datos en JSON?

`Auditoria` guarda tabla, operación y dos JSON. `vw_Auditoria` los convierte en frases: un `CASE` traduce la tabla a un módulo («Pagos de planilla»); otro `CASE`, ordenado de lo específico a lo general, decide la acción leyendo el JSON con `JSON_VALUE` (si `Anulado` pasó de false a true, «Anuló»; después los genéricos «Creó», «Eliminó», «Modificó»); el resumen sale de `fn_AuditoriaCambios` con `STRING_AGG`; sin usuario muestra `(sistema)`. `fn_AuditoriaCambios` abre los dos JSON con `OPENJSON`, los alinea con `FULL OUTER JOIN` (en un alta no hay «antes») y deja solo los campos que difieren: `SalarioBase: 600.00 → 650.00`. *(12.6, 12.11)*

10.4 Procedimientos y transacciones

P21 ★. ¿Qué se gana con un procedimiento almacenado frente a escribir el SQL en C#?

Un lugar único para las reglas: un préstamo exige unidad existente y en buen estado, sin préstamo activo, responsable real y fecha futura; en un procedimiento lo cumplen todas las pantallas, scripts y reportes. Además: parámetros con tipo (cierran la inyección), plan de ejecución guardado, operaciones de varios pasos atómicas sin que C# coordine nada y una pantalla simple (reúne datos, llama y muestra el mensaje). Honestidad: las operaciones simples (cargos, consumibles, empleados) usan `INSERT` o `UPDATE` directos con parámetros; las que tienen reglas o varias escrituras van por procedimiento. *(13.1)*

P22 ★. ¿Qué es una transacción? ¿Por qué algunos procedimientos no la tienen?

Un grupo de operaciones que se completan todas o ninguna. La salida de 50 hojas implica bajar el stock y anotar el kardex; si se corta la luz entre las dos, se contradirían para siempre; con `BEGIN TRANSACTION ... COMMIT` esa mitad no queda. ACID: **A**tomicidad, **C**onsistencia, a**I**slamiento y **D**urabilidad. Otros casos: `sp_CerrarPeriodo` (cierre más historial) y `sp_RegistrarMantenimiento` (tres escrituras; el `THROW 50051` después de dos las deshace). Una sola sentencia ya es atómica: `sp_RegistrarPrestamo` (un `INSERT`) y `sp_CambiarEstadoUnidades` (un `UPDATE` de veinte unidades) no la necesitan. La transacción se pone donde protege algo y se mantiene corta (las validaciones van antes). *(13.2, 13.3)*

P23. ¿Qué hacen `NOCOUNT`, `XACT_ABORT`, `TRY/CATCH` y `THROW`?

`SET NOCOUNT ON` no devuelve «N filas afectadas». `SET XACT_ABORT ON` marca como imposible de confirmar cualquier transacción con un error de ejecución, así que nunca queda a medias. `BEGIN TRY ... END TRY BEGIN CATCH` es el try/catch de SQL: en el `CATCH`, `IF @@TRANCOUNT > 0 ROLLBACK;` y después `THROW;` sin parámetros **relanza el**

mismo error (si no, se «tragaría» y la pantalla creería que todo salió bien). `THROW 50043, 'texto', 1;` lanza un error propio: número mayor que 50000, mensaje pensado para el usuario y un estado. C# lo recibe como `SqlException` y muestra `Message`. *(13.4, 13.5)*

P24 ★. ¿Qué problema resuelven `UPDLOCK` y `HOLDLOCK`?

La concurrencia. Hay 10 resmas y dos encargados sacan 8 a la vez: sin protección ambos leen 10, ambos pasan la comprobación, el stock queda en 2 y salieron 16 (actualización perdida). Con `WITH (UPDLOCK, HOLDLOCK)` dentro de la transacción, `UPDLOCK` dice «la leo para modificarla» y `HOLDLOCK` mantiene el bloqueo hasta el final; el segundo encargado espera, ve 2 y recibe el 50043. Mismo patrón en `sp_RegistrarDevolucion`, `sp_GuardarSalon` y `sp_CambiarEstadoUsuario` (evita dejar al sistema sin administradores). No siempre hace falta: `sp_RegistrarPrestamo` confía en el índice único filtrado, que rechazaría al segundo con un mensaje técnico. *(13.6)*

P25 ★. ¿Cómo evitan la inyección SQL?

Con parámetros: casi todo pasa por `Datos` (`Datos.P("@id", id)`), y las pantallas con su propio `SqlCommand` (el login) usan `AddWithValue`. El valor viaja separado de la consulta y se trata siempre como dato: `'; DROP TABLE Empleados; --` se guarda como texto. Los procedimientos reciben tipos concretos y procesan las listas con `STRING_SPLIT` y `CAST AS INT`; el SQL dinámico del generador de triggers usa `QUOTENAME`. Los filtros que parecen pegar texto son `RowFilter` en memoria y no tocan la base. Y las contraseñas no se comparan en SQL: son hashes verificados en C#. *(13.7)*

P26. ¿Cómo funciona `sp_GenerarPlanilla`? ¿Qué pasa al cerrar y reabrir un mes?

Primero elige candidatos (empleados no dados de baja y contratados al final del mes) en una tabla temporal `#cand` con su salario y una marca `YaTiene`. Con `@Simular = 1` devuelve cifras (pagos a generar, ya existentes, empleados sin salario) y termina: es la vista previa que usa la pantalla, que deshabilita el botón si el mes está cerrado o no hay nada que generar. Con `@Simular = 0` hace un único `INSERT ... SELECT` en una transacción, y es repetible (no duplica). Rechaza meses futuros (50802) y cerrados (50800). `sp_CerrarPeriodo` verifica que no sea futuro (50808), que no esté cerrado (50804), que tenga pagos (50805) y que ninguno esté pendiente (50803); escribe en `PeriodosPlanilla` y en su historial. Reabrir exige que esté cerrado (50806) y un motivo de al menos cinco caracteres (50807). Todo queda auditado. *(13.10, 13.11)*

P27. ¿Por qué `sp_GuardarConfiguracionInicial` usa una transacción? ¿Cómo viaja un error hasta la pantalla?

Primero comprueba que no exista configuración (50021, que cierra el asistente para siempre) y luego guarda la configuración y crea el administrador en una transacción: si fallara la creación (usuario repetido, 50022; rol inexistente, 50023) con la configuración ya guardada, el sistema creería estar configurado y no habría con quién entrar. La contraseña

llega ya como hash. El viaje de un error (salida de 50 hojas con 20 de stock): pasan las validaciones, se abre la transacción, se lee 20 con bloqueo, `THROW 50043`, el `CATCH` hace `ROLLBACK` y `THROW;`, C# recibe la `SqlException` y muestra `ex.Message`. Ni el stock ni el kardex cambiaron. *(13.12, 13.13)*

10.5 Triggers, auditoría y seguridad

P28 ★. ¿Qué es un trigger? ¿Qué son `inserted` y `deleted`?

Código que se ejecuta solo cuando ocurre un `INSERT`, `UPDATE` o `DELETE`; ideal para lo que debe pasar siempre. Los de auditoría son `AFTER`: corren dentro de la misma operación (si fallan, se cancela el cambio) y una vez por sentencia, recibiendo todas las filas juntas. `inserted` tiene las filas nuevas y `deleted` las viejas: filas en las dos es un `UPDATE` (`U`), solo en `inserted` un alta (`I`), solo en `deleted` una baja (`D`). El trigger las une por la llave con `FULL OUTER JOIN`, convierte cada fila a JSON en `Antes` y `Despues`, y si un `UPDATE` no cambió valores reales no registra nada. *(14.1, 14.2)*

P29 ★. ¿Cómo sabe el trigger quién hizo el cambio?

La conexión usa la cuenta de Windows (`Integrated Security`), así que la base ve siempre el mismo usuario. La solución es `SESSION_CONTEXT`: `Conexion.Conectar()` engancha `StateChange` y, cuando la conexión pasa de cerrada a abierta, ejecuta `sp_set_session_context @key = 'Usuario', @value = <usuario del programa>`; el trigger lee `SESSION_CONTEXT(N'Usuario')`. Sin etiqueta (Management Studio, modo Debug) queda `NULL` y se muestra `(sistema)`; los triggers de inventario caen a `SUSER_SNAME()`; cada fila guarda el `Equipo` (`HOST_NAME()`). Limitación: el `catch` vacío de C# que pone la etiqueta, a propósito, para que un fallo de auditoría no impida trabajar. *(14.3)*

P30. ¿Por qué un procedimiento genera los triggers de auditoría? ¿Qué guarda `Auditoria`?

Escribir 17 triggers casi iguales a mano invita a errores (bastaría olvidar una columna). `sp_CrearTriggerAuditoria` recibe la tabla, descubre sus columnas y su llave primaria en el catálogo y arma el `CREATE TRIGGER` con SQL dinámico (`sp_executesql`, protegido con `QUOTENAME`); omite las binarias y acepta columnas a excluir (`Clave` y `UltimoAcceso` en `Usuarios`: ni el hash se copia). Si cambia una tabla, basta regenerar. `Auditoria` responde seis preguntas —cuándo, quién, desde dónde, qué tabla, qué operación (`I`, `U`, `D`, `A`) y qué fila— más `Antes` y `Despues` en JSON con la fila completa. *(14.4, 14.5)*

P31 ★. ¿Por qué el registro no se puede borrar ni editar? ¿Por qué existen además `trg_AuditoriaUnidades` y `trg_AuditoriaActivos`?

Una auditoría editable no sirve como prueba. `trg_Auditoria_Inmutable` es `INSTEAD OF UPDATE, DELETE`: en lugar de la operación lanza el error 50900 (las altas sí pasan). Cubre errores y el uso indebido normal; no es absoluto, porque un administrador de la base podría desactivar el trigger (harían falta permisos más estrictos, copias externas o tablas *ledger*). Los dos triggers de inventario escriben en `AuditoriaUnidades` y `AuditoriaActivos` con columnas estructuradas (estado y salón anterior y nuevo) para una línea de tiempo legible de cada pieza; son selectivos y no son redundantes: dos niveles de detalle para dos públicos. *(14.6, 14.7)*

P32 ★. ¿Cómo se protegen las contraseñas? ¿Cómo es la recuperación por correo?

Nunca se guardan: `Usuarios.Clave` tiene un hash BCrypt (`$2a$11$...`), de un solo sentido, con sal aleatoria por contraseña y costo ajustable (lento a propósito). El login lee el hash con una consulta parametrizada y `PasswordHasher.Verificar` compara en C#; la base nunca ve la clave en claro. El login acepta una clave antigua en texto plano solo para migrarla, después de comprobar cuenta activa y rol. **Recuperación:** correo existente → código de seis dígitos con `RandomNumberGenerator` → se envía y se guarda solo en memoria, con vencimiento a los 15 minutos → al restablecer se revisan longitud, coincidencia, vencimiento y código; con cinco intentos fallidos se invalida (acertar es del orden de uno entre 180.000) → hash y `UPDATE` parametrizado. Mejoras: un índice único sobre el correo y un mensaje que no revele qué correos existen. *(14.9, 14.10)*

P33. ¿Qué protecciones tiene el login? ¿Cómo evitan que el sistema se quede sin administradores?

Cinco fallos con el mismo usuario bloquean el login 60 segundos (avisa los intentos que quedan desde el tercero); un acierto limpia el conteo; las cuentas desactivadas no entran; cada rechazo se audita con su causa; BCrypt encarece la fuerza bruta; y solo inician sesión los administradores aunque `Roles` tenga dos. Límite: el conteo está en memoria y es por usuario (en producción se guardaría en la base). `sp_CambiarEstadoUsuario`: nadie puede desactivarse a sí mismo (50905) y debe quedar otro administrador activo (50906), con `UPDLOCK, HOLDLOCK` para que dos administradores no se desactiven mutuamente a la vez. *(14.11, 14.12)*

10.6 Integración, preguntas trampa y cierre

P34 ★. Sigue un préstamo de principio a fin: ¿por qué capas pasa?

(1) Pantalla: campos obligatorios con aviso y selectores que solo ofrecen unidades y empleados válidos (comodidad). **(2) Procedimiento:** `frmGestionActivosPrestamos` llama a `Datos.Procedimiento("sp_RegistrarPrestamo", ...)`, que valida (50950 a 50953 y 50031 a 50035) e inserta (garantía). **(3) Tabla:** los `CHECK`, las llaves foráneas y `UX_Prestamos_UnidadActiva` (red final). **(4) Trigger:** `trg_Aud_Prestamos` registra solo

quién, cuándo y cómo quedó la fila. **(5) Vista:** la pantalla recarga `vw_Prestamos` y la `Alerta` se calcula con `GETDATE()`. Si algo falla, el error sube por la misma ruta: `THROW` o restricción, `SqlException`, mensaje en pantalla. *(15.1)*

P35. ¿Las búsquedas y filtros se hacen en la base o en el programa?

En las dos. En la mayoría de las pantallas se trae la tabla completa (`Datos.Tabla`), se envuelve en un `DataView` y se filtra en memoria con `RowFilter`: es rápido, responde al instante y es lo adecuado para cientos o pocos miles de filas. En otras se manda el `WHERE` a la vista (Préstamos: `WHERE Alerta = 'Atrasado'`). Límite: con cientos de miles de filas habría que filtrar en el servidor. Los filtros en memoria no son consultas, así que aunque lleven texto del usuario no pueden alterar la base. *(15.3)*

P36. ¿Está normalizada la base? ¿Dónde rompe las reglas a propósito?

En lo esencial, sí. **1FN:** una celda, un valor (el docente no tiene una celda «Matemáticas, Física»: son filas de `EmpleadoMaterias`). **2FN:** `EmpleadoMaterias` solo tiene las dos columnas de la llave. **3FN:** el nombre del cargo vive en `Cargos` y el del estado en `Estados`. Desviaciones deliberadas: `PagosPlanilla` copia base, bono y descuento (el historial del mes no cambia), y `Prestamos` conserva el nombre del responsable (un alumno no tiene tabla) y el código de inventario. Regla: se normaliza para evitar contradicciones al guardar y se repite cuando el dato es una fotografía histórica. *(15.4)*

P37. Preguntas trampa de diseño: ¿qué pasa si renombran el cargo «Docente»? ¿Por qué el nombre del empleado es único?

Varias reglas reconocen las cosas por su nombre: un docente es quien tiene un cargo que empieza con «Docente» (`LOWER(NombreCargo) LIKE 'docente%'`), los procedimientos comparan con `'Dado de baja'` y el estado Inactivo es el 2 en seis lugares. Si cambiara el texto, la regla fallaría sin avisar. `frmAsignarCargo` ya impide que un cargo deje de empezar con «Docente» y los estados son un catálogo fijo; la mejora es un código estable en el catálogo o funciones como `fn_IdBodega`. `UQ_Empleados_Nombre` evita altas duplicadas, pero dos personas con el mismo nombre no podrían coexistir: el identificador real es el DUI (`UX_Empleados_DUI`), y lo lógico sería confiar en él para la unicidad. *(15.5, 15.6)*

P38. ¿Qué pasa si se va la luz? ¿Se puede usar desde varias computadoras? ¿Cómo se respalda?

Luz: la base queda como antes de empezar; la generación es una transacción y SQL Server, al reiniciar, rehace lo confirmado y deshace lo no confirmado desde su registro de transacciones; se puede volver a generar sin duplicar. **Varias computadoras:** tal como está (LocalDB) es una sola; para red hace falta una instancia de SQL Server (Express es gratuita) y cambiar la constante `Servidor`, y la parte difícil de la concurrencia (bloqueos, transacciones, índices filtrados) ya está resuelta. **Respaldo:** `BACKUP DATABASE`

`Gestion_administrativa_y_escolar TO DISK = N'C:\Respaldos\Gestion_administrativa_y_escolar.bak' WITH INIT, CHECKSUM;` y, con el programa cerrado, `RESTORE DATABASE ... FROM DISK = ... WITH REPLACE;`. Prueba una vez la restauración: un respaldo que nunca se restauró es una promesa. *(15.7 a 15.9)*

P39. ¿Cómo demuestro que la base se defiende sola? ¿Qué pasa si presento en Debug?

Con las seis consultas de la sección 11.3 en Management Studio, narrando qué prueba cada una: vista (la alerta se calcula sola), función (22 días hábiles), `CHECK` (falla con el tipo 'Cocina'), trigger (falla con el 50900), procedimiento (falla con el 50514) y `vw_Auditoria` (todo queda legible). Orden: lo que se ve, lo que protege, lo que vigila y la prueba de que quedó registrado. Antes de presentar, cambia a **Release**: en Debug el programa salta el login, no podrás mostrar el bloqueo ni la recuperación y los cambios saldrán como `(sistema)`. Para mostrar el asistente de primer uso hace falta una base nueva o una copia anterior a la configuración. *(15.10, 15.12)*

P40 ★. ¿Qué mejorarías con más tiempo?

Ten una lista preparada, de menor a mayor esfuerzo: un índice único sobre el correo de `Usuarios`; que el `SalarioNeto` de `Planillas` sea calculado; pasar las operaciones simples a procedimientos; centralizar las reglas de baja en un procedimiento auxiliar; guardar el bloqueo del login en la base; reemplazar textos o números sueltos (cargo «Docente», estado 2) por identificadores estables; una tabla de feriados para `fn_DiasHabiles`; y, para uso en red, una instancia de SQL Server con usuario propio. Mencionar tres o cuatro con su motivo transmite que el límite del sistema es de alcance, no de comprensión. *(15.11)*

11. Glosario mínimo y repaso final de 20 minutos

Esta sección es para el final: el glosario te sirve para comprobar que cada palabra técnica del script te suena a algo concreto, las frases ancla y los límites son lo que debes decir tú antes de que te lo pregunten, y la demostración y el repaso cronometrado cierran el estudio.

11.1 Glosario mínimo: las 35 palabras que debes poder explicar en una frase

Término	En una frase
Tabla, fila y columna	Una tabla guarda un tipo de cosa; cada fila es un caso concreto y cada columna una característica con su tipo

Término	En una frase
Llave primaria (`PRIMARY KEY`)	La columna que identifica cada fila sin ambigüedad: no se repite ni queda vacía
Llave foránea (`FOREIGN KEY`)	Columna que apunta a la llave primaria de otra tabla; el motor impide referencias a algo que no existe
Integridad referencial	Garantía de que las referencias entre tablas siempre apuntan a filas reales
`IDENTITY(1,1)`	El motor numera solo, sin repetir ni reutilizar números
`CHECK`	Condición que cada fila debe cumplir; solo rechaza cuando da falso (con `NULL` pasa)
`UNIQUE`	Ningún valor se repite en esa columna
Índice único filtrado	`UNIQUE` con `WHERE` : la unicidad cuenta solo para las filas que cumplen el filtro (`UX_...`)
Columna calculada `PERSISTED`	La calcula y guarda el motor (`SalarioNeto`); nadie la escribe
Normalización	Guardar cada dato en un solo lugar; se rompe a propósito solo para guardar fotografías históricas
`ON DELETE CASCADE`	Al borrar el padre se borran solas las hijas; solo en datos sin sentido propio
Vista	Consulta guardada con nombre; no guarda datos y está siempre al día
Función (escalar o de tabla)	Calcula y devuelve un valor o una tabla; se usa dentro de otra consulta y no modifica datos
Procedimiento almacenado	Programa guardado en la base que ejecuta una operación con sus reglas y su transacción
Parámetro (`@x`)	Hueco cuyo valor viaja aparte del texto; previene la inyección SQL
Transacción (`COMMIT` / `ROLLBACK`)	Grupo de operaciones que se confirman todas o se deshacen todas
ACID	Atomicidad, Consistencia, alslamiento y Durabilidad

Término	En una frase
`SET XACT_ABORT ON`	Cualquier error dentro de una transacción la cancela entera
`TRY / CATCH` y `THROW`	El try/catch de SQL; `THROW;` relanza el error y `THROW 50043, 'texto', 1;` lanza uno propio
`UPDLOCK, HOLDLOCK`	Bloquean la fila leída hasta el final de la transacción para que dos usuarios no la modifiquen a la vez
Actualización perdida	Dos personas leen el mismo valor y la segunda pisa a la primera
Trigger	Código que se dispara solo con un `INSERT`, `UPDATE` o `DELETE`
`AFTER` / `INSTEAD OF`	El trigger corre después del cambio, o en lugar de él (para rechazarlo)
`inserted` y `deleted`	Tablas temporales del trigger con las filas nuevas y las viejas
`SESSION_CONTEXT`	Datos pegados a la conexión; así el trigger sabe qué usuario hizo el cambio
JSON (`FOR JSON`, `OPENJSON`)	El formato en que se guarda el «antes» y el «después» de la auditoría y cómo se lee de vuelta
Baja lógica	Cambiar el estado o una bandera en vez de borrar, para conservar el historial
Kardex	El registro de cada entrada y salida de un consumible, con el saldo resultante
Idempotencia	Ejecutar el script otra vez no rompe nada ni borra datos
Lote y `GO`	`GO` no es SQL: es la orden de la herramienta que separa el script en lotes
Esquema `dbo`	La «carpeta» lógica por defecto de los objetos (`dbo.Empleados`)
SQL dinámico y `QUOTENAME`	Armar y ejecutar SQL desde texto (`sp_executesql`), con los nombres protegidos
Hash, sal y BCrypt	Función de un solo sentido; valor aleatorio por contraseña; hash lento con sal y costo ajustable

Término	En una frase
Inyección SQL	Meter código SQL por un campo de texto; se evita con parámetros
LocalDB	SQL Server liviano sin servidor, con autenticación de Windows (`Integrated Security`)

11.2 Seis frases ancla

Son ideas completas que puedes decir con tus palabras; si las recuerdas, el resto de la explicación sale sola.

- «Las reglas viven en la base de datos, la pantalla solo las pide y las muestra.»
- «Cada pieza física es una fila propia; el catálogo solo dice qué tipo de cosa es.»
- «Una operación que toca varias tablas va en una transacción: todo o nada.»
- «La alerta no se guarda, se calcula al mirar.»
- «El historial lo escriben triggers, así que nadie puede olvidarse de registrar un cambio.»
- «Hay tres barreras: la pantalla, el procedimiento y la restricción de la tabla.»

11.3 Demostración en vivo en Management Studio

Mostrar que la base se defiende sola impresiona más que describirlo. Estas consultas no dañan datos: o solo leen o están pensadas para fallar. Pruébalas antes en tu equipo y ajusta los números a los que existan en tu base.

```
-- 1. Una vista: la alerta se calcula sola
SELECT TOP 5 IdPrestamo, Activo, FechaLimite, Alerta FROM dbo.vw_Prestamos
ORDER BY IdPrestamo DESC;

-- 2. Una funcion: octubre de 2026 tiene 22 dias habiles
SELECT dbo.fn_DiasHabiles('2026-10-01', '2026-10-31') AS DiasHabiles;

-- 3. Una restriccion CHECK: 'Cocina' no es un tipo de salon valido (debe
fallar)
INSERT INTO dbo.Salones (IdSalon, NombreSalon, Tipo) VALUES (9999, 'Prueba',
'Cocina');

-- 4. Un trigger INSTEAD OF: el registro de actividad no se modifica (debe
fallar con 50900)
UPDATE dbo.Auditoria SET Usuario = 'otro' WHERE IdAuditoria = 1;

-- 5. Una regla de negocio en un procedimiento: pedir mas unidades de las que
hay (debe fallar con 50514)
EXEC dbo.sp_AsignarUnidades @IdActivo = 1, @IdSalonDestino = 2, @Cantidad =
100000;

-- 6. El historial legible
SELECT TOP 5 FechaHora, Usuario, Modulo, Accion, Resumen FROM
dbo.vw_Auditoria ORDER BY IdAuditoria DESC;
```

Cada una ilustra una pieza distinta: vista, función, restricción, trigger, procedimiento y auditoría. Dos deben fallar, y eso es justo lo que quieres enseñar: el mensaje de error es la prueba de que la regla funciona. Narra antes de ejecutar qué vas a probar.

11.4 Límites del diseño: dilos tú antes de que te los pregunten

- Las operaciones simples (cargos, consumibles, baja lógica de empleados) usan `INSERT` o `UPDATE` directos con parámetros; solo las operaciones con reglas pasan por procedimientos.
- La regla de la baja está repetida en cuatro procedimientos → un procedimiento auxiliar la centralizaría.
- Varias reglas reconocen cosas por su nombre (el cargo «Docente», «Dado de baja») o por un número (estado 2) → identificadores estables.
- El nombre del empleado es único, pero el identificador real es el DUI; el correo único lo garantizan los procedimientos, no un índice → un índice único sobre `Usuarios.Correo`.
- Para el DUI, la base valida solo el formato (`CK_Empleados_DUI`); el dígito verificador se calcula en C#.
- `SalarioNeto` es calculada en `PagosPlanilla` pero columna normal en `Planillas`.

- `fn_DiasHabiles` no conoce los feriados, y las funciones escalares se ejecutan fila por fila (adecuado para la escala de una escuela).
- La auditoría es inmutable para el programa, no contra un administrador de la base; los cambios hechos desde Management Studio salen como `(sistema)`.
- El bloqueo de intentos del login está en memoria y la recuperación revela qué correos existen.
- LocalDB sirve a una sola computadora; el filtro en memoria es para cientos o pocos miles de filas; siete vistas de resumen existen pero ninguna pantalla las consulta.

11.5 Repaso final de 20 minutos

Hazlo en voz alta y con cronómetro, sin mirar la guía; lo que te cueste, márcalo y vuelve a leerlo al final.

1. **3 min, el mapa.** Di en una frase qué hace cada una de las cinco piezas y cómo se reparte la lógica en tres barreras (sección 2).
2. **4 min, dibujo.** Dibuja de memoria `Empleados` con `Cargos`, `Planillas`, `PagosPlanilla` y `EmpleadoMaterias`, y `UnidadesActivo` con `GestionActivos`, `Salones` y `Prestamos` (sección 3).
3. **4 min, el procedimiento.** Explica `sp_RegistrarMovimientoConsumible` en siete partes y por qué existen `UPDLOCK, HOLDLOCK` (sección 5).
4. **3 min, el trigger.** Cuenta cómo se registra un cambio y quién lo hizo: `inserted` / `deleted`, `SESSION_CONTEXT` e `INSTEAD OF` (sección 6).
5. **3 min, un flujo.** Narra el préstamo capa por capa: pantalla, procedimiento, tabla, trigger, vista (sección 8).
6. **3 min, límites y mejoras.** Di la lista de la sección 11.4 y las tres mejoras que harías primero (P40).

11.6 Antes de presentar: lista de comprobación

- Cambiar a la configuración **Release** y comprobar que el programa pide usuario y contraseña.
- Hacer una copia de seguridad justo antes de presentar, con los datos como quieres mostrarlos, y haber probado una vez la restauración. Para el asistente de primer uso, tener a mano una copia sin configurar.
- Ejecutar antes las seis consultas de la demostración (sección 11.3) y ajustar los números a tu base.
- Ejecutar `SELECT Usuario FROM Usuarios WHERE Clave NOT LIKE '$2%'` y confirmar que no devuelve filas (todas las claves ya están en hash).

- Tener abierto `BaseDeDatos_Completa.sql` con la búsqueda (Ctrl+F) lista para `Empleados`, `sp_RegistrarMovimientoConsumible` y `trg_Aud_Empleados`.
- Si te preguntan algo que no sabes: reconocerlo con precisión, razonar con lo que sí sabes y abrir el script con el profesor; nunca inventar.